

The single source of truth for what each model **is**, **how it works**, its **honest current status**, and **where the code lives**.

***How this file went wrong, recorded because it caused real damage.** Between 2026-07-28 and 2026-07-30 it was not updated while ~40 commits landed, and a session then mischaracterised the whole model family from it: **DODUO** described as unbuilt when it had been built, wired, controlled and measured at 42.0%; **MEDICHAM** audited in its superseded v2 file, with that file's limitations reported as ABRA's; top-K pruning "proposed" that `engine/fit_joint.js` already implemented. Two rules came out of it — read the implementation and check which file a consumer actually requires, and never report a red test as a "known failure" (`CLAUDE.md`).*

Which files actually play a game: `engine/mew.js` loads `engine/magnemite.js` (MAG), and `engine/board.js` loads `engine/medicham2-browser.js` for damage. That is the entire live path. Everything else on this page is off it — which does **not** mean dead, only that "is this live" must be answered by grepping for consumers rather than by file date. Superseded implementations live in `engine/graveyard/` and are not to be fixed, cited, or audited as current.

Guiding principle: **garbage in, garbage out**. The browser engine's **damage math is now validated** against the Smogon damage calculator (within 5% on 100% of tested scenarios — see MEDICHAM below). The remaining GIGO caveats are the rollout *policy* and format-specific data (Champions rule changes, some Mega/ability specifics) — stated per-model.

Job: score the PAIR of choices, not two choices separately. Named for the two heads on one body: two slots, one decision. 18 coordination features — `focusFireKills`, `redirectThenAttack`, `boostsPartnerDamage`, `speedSetupHelpsPartner`, `weatherSetupHelpsPartner`, `healsPartner`, `doubleKO`, `flinchThenSetup`, `screenWhileThreatened`, `spreadFreeBesideAlly`, and the rest. **Why it matters, and it is a strategic argument not a tidiness one (Will's):** a team must optimise for TEAM success, not individual Pokémon success. A policy that picks each slot independently can be set positions that REQUIRE coordination and will fail them every time — a repeatable hole rather than variance, and precisely what WOBBUFFET searches for. **I retired this earlier and was wrong.** The retirement rested on the double-target rate — MAG 24.6% against humans 23.2%. That metric touches **2 of the 18 features**. Judging a team-coordination model by a targeting statistic is judging a ninth of it. **TRAINABLE FOR WINNING SINCE 2026-07-31** (`ce5367c`). Every number below this line was fitted to predict a human click, including all 18 pair terms — the objective this project has measured as the binding constraint twice. `train_policy.js --joint` now moves the pair block by whether the game was **won**. The gradient of the pair softmax is the concatenation `[xa + xb, jf]`: the two single vectors summed (both are scored by the same single block `ws = wj.slice(0, 56)`), then the 18 pair terms. Vector length **74 = 56 + 18**.

Wiring notes that matter to anyone touching it: `--joint` is **per arm**, so a training run needs `--joint --joint2` and `mew.js` refuses `--learn` without the pair; each iteration's checkpoint reaches the players through `--joint-weights`, because magnemite otherwise re-reads the frozen `data/policy-weights-joint.json` and the pair terms never move; `preflight.js` reports `joint` (pair terms) as its own block, so a dead coordination layer reads as one cause rather than 18.

*Status: wired and gated, **not yet measured for winning**. Do not quote a win rate for trained DODUO — none exists.* First evidence it moves at all: two iterations at 40 games each put `bothSameTarget` **+0.164** (third-largest change in the whole vector), `overkill` **+0.120**, `focusFireKills` **+0.094** — self-play wants focus fire more than the human fit did.

Refitted 2026-08-02 on `engine/click_match.js` — 7,454 clean games, **81,515 usable joint turns** (66,236 before; 24,997 before the spread-matcher fix), 66,520 train / 14,995 held out:

predicting which PAIR a human clicked	log-lik	top-1
two moves decided separately (what MAG does)	-3.3425	10.1%
refitted, joint terms forced to zero	-3.3318	9.4%
with the joint terms	-3.2447	12.2%

Read the middle row before the last: refitting the single-move weights on pair data buys nothing on its own here, and **the whole gain belongs to the coordination terms**. (At 48 features on 2026-07-28 the split was the other way round — over half the gain was the refit — so this reads differently now that the matcher is not discarding a quarter of the turns.) All of it predicts a human click; it is not evidence the pair wins more games.

Stability, and it is the answer to a question that was open: the 2026-08-01 refit flipped **nine of eighteen** pair signs, which looked like an unstable block. Handed a further 15,279 turns, this refit flips **none of 74** weights and moves the vector 12.0% in L2. `engine/collinearity_joint.js` says why: the highest VIF in the pair block is **2.2** and no other exceeds 1.7, so there is no credit-splitting to destabilise it. The nine flips were the matcher fix changing what the data said, not noise.

What the audit did find is the opposite failure — three weights fitted on almost nothing:

`terrainSetupHelpsPartner` carries the block's **largest** coefficient (+1.605) and fires on 0.00% of enumerated alternatives; `weatherSetupHelpsPartner` 0.04%; `boostMayConvertKill` 0.06%. `data/collinearity-joint.json` records the fire rate beside every weight, because in a table of coefficients a barely-observed one looks identical to a well-supported one. **Now wired into** `magnemite.js` **for the first time** (`--joint` , off by default). It had never once been in the loop, so the project had never tested whether coordinated choice helps; retiring it would have closed a question that was never opened. **The wiring bug worth remembering:** the first smoke test decided **0 pairs and fell back 99 times** while reporting nothing wrong — 0 games discarded, no error. The partner's options were read from the raw request rather than the reshaped list `chooseMove` receives, so every partner candidate parsed as unusable. A head-to-head run at that point would have returned ~50% and I would have reported that coordination does not help. It was caught only because the fallback is COUNTED and printed. Fixed: 100% of eligible turns now decided as a pair. **MEASURED 2026-07-28, AND IT LOSES.** Coordination ON against coordination ZEROED — the entire pair path either way, same single weights, same top-K cap, same softmax over pairs, so the only difference is the 18 coordination weights. 2,000 seed-paired games, harness fair at 49.3%:

	result
unpaired win rate, coordination ON	42.0% [39.9, 44.3] over 1,934 games
decisive pairs, coordination ON wins both	28.4% [23.9, 33.3] of 356

Not close, well powered, and consistent across every cut. It loses hardest in short games (22.0% under 8 turns) and when it does not draw first blood (14.1%), which is a bot giving away tempo. It KO's less (22.3% against 25.1%) and Protects nearly twice as often (1.74% against 0.93%).

Why, and it is the same lesson MACHAMP taught. These are IMITATION weights. The fit prices `spreadFreeBesideAlly` at -5.054, `terrainSetupHelpsPartner` at -3.989 and `screenWhileThreatened` at -3.372, at $\lambda = 0$. Those are statements that humans rarely click those pairs, not that the pairs are bad — and a bot told to avoid a free spread move beside its own ally by -5 will decline its best plays. Predicting a human pair (14.5% top-1, up from 5.9%) and winning are different objectives, and this is the cleanest separation of the two the project has measured.

WITHDRAWN 2026-08-01. All three of those numbers were a fitter defect, not a preference.

`fit_joint.js` required the candidate's target to match the human's recorded target, and a spread candidate is built with `targetMon: null` because Earthquake is not aimed — so **no spread click could ever match**. Spread moves are 14.94% of all human move clicks and 1,393 of 1,397 were discarded; the fit ran on 24,997 of 82,483 joint turns, and the missing 70% was precisely the turns these three features describe. Refitted on 63,305 turns, all three change sign: `spreadFreeBesideAlly` $-4.986 \rightarrow +0.863$, `terrainSetupHelpsPartner` $-4.125 \rightarrow +2.005$, `screenWhileThreatened` $-2.982 \rightarrow +0.110$. The corrected vector then **beat the shipped one at 66.7% and 65.9% of decisive pairs** on two disjoint seed blocks. So the paragraph above has it backwards for these three: the imitation fit was not expressing a human preference against good play, it was never shown the plays. **The broader claim is untouched and still stands on its own evidence.** Imitation and winning ARE different objectives — greedy action selection is worth about 12 points, and the resemble-vs-win table further down shows real sign flips in `overkill`, `focusFireKills` and `partnerCoversMe`. What is retired is these three features as the illustration of it, and DODUO's 42.0%, which was measured on the contaminated vector and does not describe the current one.

What this does NOT settle. Will's argument was about EXPLOITABILITY — that a bot choosing each slot independently can be set positions it fails every time. That is a claim about the worst case against a prepared opponent, not about the average, and a policy can be worse on average while being harder to counter. Nothing here tests it. `engine/exploit.js` now accepts `--target <weights.json>`, so WOBBUFFET can be pointed at DODUO for the first time. Until that runs, the coordination question is open, not closed.

The coordination features are not refuted either — the imitation fit of them is. Refitting the 18 pair weights for WINNING rather than for resemblance (MACHAMP over the joint vector) is the untested version of this idea.

UPDATED 2026-07-30 — this is now roadmap item 1, and the gap is exact. The evidence for it got much stronger: four knowledge additions produced four nulls that day while two objective changes produced two large wins (see MAGNEMITE). DODUO has only ever been fitted to the losing objective. The remaining work is wiring, not a new model:

- `engine/train_policy.js` has **no joint support** at all.
- `magnemite.js` 's learning gradient is sized to `this.w` (53 single weights), while the joint vector is `this.wj` (53 + **21** pair weights — the list grew from 18). So self-play training cannot reach the coordination weights.
- The pair softmax is the same conditional logit, and `accumulateLogitGrad(g, vecs, probs, j, nW)` is already generic over vector length.

Do choice-lock first. `fit_policy.js` hands `candidates()` all four sheet moves with no legality filter, so a choice-locked human appears to have had ~9 options when they had 4 — the logit denominator contains actions that were never available, on **6.52%** of items. Both DODUO arms inherit that error today.

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost **31.2%** on decisive pairs and would have been reported as "coordination does not help."

Already implemented, do not rebuild it: top-K capping by single-move score, keeping the human's chosen pair regardless of rank so the fit cannot manufacture agreement, and reporting how often the chosen pair falls outside K. `--joint-zero` is a true control (whole pair path, coordination weights zeroed) and `stats.jointFellBack` counts degradation.

Measured cost of the pair path, on a real mid-game board (2026-07-30): $9 \times 8 = 72$ joint actions per side. Only **28 of 72** have a non-zero joint vector — the other 44 score exactly as the sum of two singles already computed. With two Pokémon the coordination graph is a single edge, so the MARL factorisation

machinery (QMIX, QPLEX, Max-Plus) is unnecessary; what that literature does contribute is the reason independent scoring fails, since a monotonic factorisation cannot represent "Protect while my partner removes the threat." **Code:** `engine/fit_joint.js` → `data/policy-weights-joint.json` ; played via `--joint` in `engine/mew.js` .

Job: make MAG stronger by WINNING, not by resembling people. **Method:** champion/challenger hill-climb over MAG's policy weights. Candidates are perturbations of the current champion; each plays the champion over hundreds of seed-matched games and is promoted only when a Wilson interval clears 50%. The opponent is the CURRENT champion, so the bar rises every generation — hill-climbing against a frozen target produces a policy that beats that target and nothing else, which this project already fell for once. **Why it matters more than anything else on the list:** every other model here is fitted to PREDICT A HUMAN CLICK. That is a ceiling, and it is measured: re-optimising the same features for winning moved the kill proxy from +0.34 to +2.75, an eightfold change on exactly the signal the imitation fit throws away. MACHAMP is the only component whose objective is the thing actually wanted. **Honest status: half-run and stale.** The 2026-07-26 run completed **2 of 6 generations on a 17-FEATURE vector and recorded no verdict.** The vector is now 48. Re-running it is the single largest untested lever in the project. **Guards worth keeping:** every promoted champion is played against EVERY previous generation, not just the one it displaced, so that "gen 5 beats gen 4" is never mistaken for progress on its own. The guard stays — it is cheap, and it is the only thing that would *detect* a cycle. Its old justification did not: it read "this metagame is cyclic", and that is **withdrawn 2026-08-02.** `data/slowking-playstyle-eval.json` rates its own strongest cycle supported: `false` (the best of 336 candidate triples, with legs resting on as few as 5 games), and greedy-minus-Nash is 0.026 with a CI of [-0.0001, 0.1498]. Non-transitivity here is *unestablished*, which is not the same as *absent* — the guard is insurance against it, not evidence for it. `tests/test-docs-current.js` re-reads that artifact on every run and will license the claim again by itself if the metagame ever supplies it.

*Those four figures are correct and the file they name currently disagrees with them. 2026-08-04. `data/slowking-playstyle-eval.json` on disk is a byte-identical copy of `data/slowking-eval.json` — a GURU species-pair run written under the playstyle filename on 2026-08-03 15:15, because `engine/slowking_preview.py` takes its output name from `TAG` and its matrix from `MATRIX_FILE`, which defaults to GURU. It reads 1,320 triples and gap 0.0409 [-0.0001, 0.1735], not 336 and 0.026 [-0.0001, 0.1498]. Re-running with both variables set reproduces this ledger's numbers exactly, so **the withdrawal above still stands on measured evidence** — it is the artifact that needs repairing, not the claim. See the SLOWKING entry for the full finding. Any check reading that file today is reading a GURU result.*

Limit, stated: it searches the weight vector, not a policy space. It cannot learn anything the feature set cannot see, and after 2026-07-28 we know the feature set is the binding constraint for a static model. **KEPT, 2026-07-30** (Will, reversing an earlier call to graveyard it). The **artifact is gone, deleted 2026-08-02** — it was a **48-feature** vector against today's **56**, trained under the broken mega handling, with no consumer anywhere in the repository. Recorded here rather than silently removed, because a champion vector that no longer matches the feature set cannot be loaded and keeping it invited someone to try; `git show 5264585^:data/policy-weights-machamp.json` recovers it. The **method is alive and has a successor:** `engine/train_policy.js` implements the same win-objective idea by policy gradient over self-play, and is the thing that measured 55.9%. Re-running MACHAMP on the current vector is roadmap item 4. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, so that "gen 5 beats gen 4" is not read as progress by itself. (Kept as insurance, not as evidence — see the withdrawal above.) **Code:** `engine/ladder.js` → `data/ladder.json` . Companion: `engine/brood.js` (how many candidates a generation can actually tell apart).

RECONCILED 2026-07-31. *That 55.9% was measured on the 53-feature vector with switching OFF. Repeating the experiment on the 56-feature vector with switching ON gives 48.1% [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

Job: how readable is MAG? Build the bot whose only purpose is to beat it, and see how badly it wins.

Method: hill-climb over MAG's OWN feature weights, maximising win rate against MAG. Named for Counter/Mirror Coat — whatever you do, it returns the thing that beats it. **Honest status: THERE IS NO**

EXPLOITABILITY NUMBER. 2026-08-04. The former status read "stale, and its result is the most important number in the repo" on the strength of a counter that beat MAG ~63.2% [56.6, 69.3], mirror control 47.5%~. **That is retracted:** 17 features against the 58 we ship, an engine 25 wire-fixes old, computed **before the quality filter existed** — which is why `provenance.js` carried it as its only `UNSAFE` artifact. **The re-run that was meant to replace it is VOID.** It ran at full size (220 games × 24 rounds + 1,600 held-out games) and `data/policy-weights.json` — the defender — **was refitted at 22:15:24 UTC while it was running**, `engine/board.js` was written mid-search, and `engine/medicham2-browser.js` changed content twice more after it, sampled 90 seconds apart. Its figures are not quotable and are recorded struck through in `docs/SEARCH.md` §R8. **Two things survive, and both are about the tool rather than about MAG.** The hill-climb accepted **1 of 24** steps (against 10 of 18 in the 17-feature run) and its step scale decayed to 0.0168, so from round ~10 it was perturbing a near-copy of MAG — **the attack dies in 58 dimensions and would have returned an uninformative null on a still tree too.** And

`provenance.js` now marks `exploitability.json` `ok`, falsely: the artifact is 153 s newer than the weights file but was computed from a version of it 34 minutes older, and an mtime check cannot see that. **So MAG's readability is UNMEASURED, not merely stale.** `docs/SEARCH.md` §R8 has the timeline, the corpus, the five defects in `engine/exploit.js`, and the prepared re-run with its preconditions. **Read it with MACHAMP:** MACHAMP raises the bar, WOBBUFFET measures how easily the bar is cleared. Together they are the win-objective loop; separately neither means much. **Caveat on the metric itself (2026-07-28):** exploitability grades "can a prepared opponent read us", which assumes an adversary that studies you over many games. A tournament opponent has never seen you play. It is a real number and it is not the same as "do we win", which has never been measured against a human at all. **BACK ON THE ACTIVE LIST, 2026-07-30** (Will: "ADD WOBBUFFET BACK TO THE LIST"). Now **three** feature-generations stale (17 → 53), and it is roadmap item 3 because it is the only measurement here that is not bots grading bots on average — it grades *readability*. A policy can improve on average and stay exactly as exploitable; those are different numbers and only one of them has been moving. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, which is the only way to test the exploitability argument the 42.0% result explicitly does **not** settle. (*That pointing has still not happened — the 2026-08-04 re-run measured the shipped vector only, and pointing a one-step search at DODUO would produce a second uninformative null.*) **Code:**

`engine/exploit.js` → `data/exploitability.json`; the held-out confirmation is `data/exploitability-holdout.json` and its generator does **not** live in `engine/`, so `provenance.js` does not enumerate it. Folding it into `exploit.js --confirm` is the fix.

Job: instant pre-game win probability from two team sheets. **Method:** Bradley-Terry-style logistic — sum of learned per-species strengths + speed/firepower edges + a team-vs-team type-coverage term, through a sigmoid. Rarity-aware L2 shrinkage; recency-weighted. **Honest status: demoted to a fast prior, not an oracle.** Held-out backtest (`eval_harness.py`): accuracy ~55% (at the format's skill ceiling) but **log-loss ties a coin** (0.699 vs 0.693) and only matches player-Elo. Overconfident (temperature ≈ 6 to calibrate). The team sheet simply doesn't determine the winner much in a non-transitive, high-variance format — that's a real

finding, not just a weak model. **Code:** `engine/jolteon.py` (train), `pwin()` in `web/index.html` (deployed). **RETIREMENT WITHDRAWN 2026-07-30 — it was retrained and it works.** The old verdict below was reached on a model trained through the raw ladder store, where `jolteon.py` defined its own `load_games()` and read a population that is ~87% bots, forfeits and stubs. Retrained on **self-play** (`JOLTEON_SELF` ; `JOLTEON_RAW=1` restores the old behaviour), it moved from worse-than-a-coin to genuinely predictive. It is a real input to DITTO again rather than a candidate for deletion.

But it is ADDITIVE, and that is DITTO's binding problem — 258 species weights plus 2 extras (speed, damage), with no pairwise term. Measured 2026-07-30: the additive species block can move the score by ~6.3 while speed and damage together move it by at most ~0.4, so hill-climbing it converges on the six highest-weighted species. **It cannot represent that Pelipper + Archaludon is worth more than Pelipper plus Archaludon** — the same expressiveness failure as DODUO, one level up. See DITTO.

The old verdict, kept because a prior conclusion is never silently rewritten. Every preview-level model in this project sat at the same coin-flip ceiling — JOLTEON, preview roles, CHOMP-EV, and as of 3.2.0 WAR. The 2026-07-25 finding that the 55% skill ceiling was itself bot-contaminated (52.4% clean, CI includes 50) makes the ceiling lower than JOLTEON was built against, not higher. A low-rank non-transitive term would not change that. It survives only as a candidate shortlist for DITTO; it should make no win-probability claim anywhere.

MECHANICS STATE, 2026-08-05 (3.41.0), read from the artifact rather than typed:

`data/mechanics-census.json` reads **202 live of 205 probed, 3 missing, 0 hollow** (wires 82–89 at 3.40.0, then the Layer 0 pass — wires 90–112 — at 3.41.0; Marvel Scale and After You/Quash came off the missing list, the second because the "cannot tell it from Instruct" blocker was false: Instruct carries `instructsTarget {extraAction:true}`, a shape read). The three remaining are each declared with a reason in [ENGINE.md](#) — two on the `moveAccuracy(id, field)` signature (a change that crosses into the refit edge MEASURE owns), one by DECISION (Avalanche asks for turn state `dmgRange` is not given).

THE INTERACTION MATRIX IS NOW A SEPARATE, GENERATED CLAIM and it is the one that says whether the mechanics work TOGETHER. `data/interaction-matrix.json` : a theoretical cross product of **8,506** carrier x reactor pairs, **1,640** emitted after a named and counted co-occurrence filter, **1,008** of those LIVE by the reference engine's own two-arm test, and medicham2 matches the official pinned Showdown engine on **93.3%** of them. The multi-turn field axis — every ordered pair of persistent field effects, run eight turns to expiry — is **156/156**. Ten engine bugs were found by it in one pass (WIRE 72–81); none was reachable from a single-mechanic probe. **Job:** grounded win rate by actually playing the matchup out.

Method: real Gen-9 **doubles** Monte-Carlo rollout (`engine/medicham2-browser.js`, embedded as `MEDI2` in the site). Damage formula with boosts, spread $\times 0.75$, crit, rolls, STAB, type, weather, Trick Room, Tailwind, priority, Protect, items (scarf/band/specs/AV/Life Orb/leftovers/sitrus/**Expert Belt/Muscle Band/Wise Glasses**), and a **validated ability/item layer** (Ruin quartet, Solar Power, Guts, Orichalcum Pulse, Hadron Engine, Adaptability, Technician, Tinted Lens, Filter/Solid Rock, Multiscale, Thick Fat, Heatproof, Purifying Salt, type-immunity abilities). **Mega abilities tracked** (base vs Mega stone: Staraptor→Contrary, Swampert→Swift Swim, + canonical Megas). Status, Fake Out flinch, **recoil, self-stat-drop moves with Contrary flip, weather-speed abilities** (Swift Swim etc.). Policy = **behaviour cloning** (samples the move real players click) + take an obvious KO + need-based Protect, now **accuracy-weighted** (a 70% nuke isn't a guaranteed KO) and recoil-aware (reduces the fast-frail over-crediting). **Win% backtest — RE-MEASURED 2026-08-04 against the current engine, on 6,886 clean games.** The 2026-07-23 and 2026-08-02 readings below are kept because a prior conclusion is never silently rewritten, but neither should be quoted: both scored `winProb2`, which **no live decision calls**. MILTANK's team-preview leaf is a greedy payout at `maxTurns=60 / seeded:true`; its in-game leaf is `rollout_leaf.rolloutWinProb` at `explore=1.0 / foePolicy=uniform / maxTurns=60`. Scored on identical turn-0 positions from real brought teams, both **lose to a coin**, paired: in-game leaf Brier +0.0502 (95% CI 0.0371 to 0.0628) over 1,378 held-out games at 200

rollouts; preview leaf +0.0740 (0.0668 to 0.0813) over the full 6,886. Both also lose to player-Elo on the 4,906 rated games. The reliability curve is nearly **flat**: the in-game leaf's 90-100% bucket wins 53.6% and its 0-10% bucket wins 53.8%, and it names the winner on 50.99% of 1,314 decisive calls (CI 48.3-53.7, $p=0.47$) — no discrimination at all. The preview leaf does discriminate, barely: 53.22% of 6,700 (CI 52.0-54.4, $p<1e-4$), about 1.9 points above its own split-half noise floor, while putting 25.6% of its predictions into the two extreme buckets where it is wrong by ~40 points. **So the win% is a weak ranker and not a probability, and the search is maximising the region where it is most wrong** (LESSONS 2). Harness:

`engine/backtest_winrate.js` ; report `data/winrate-backtest.json` (which stamps the sha256 of every engine source it was measured against) plus per-game rows in `data/winrate-backtest-rows.jsonl` .

The 2026-07-23 reading, superseded, kept: on 600+ held-out real games, MEDICHAM's raw $P(\text{win})$ **does not beat a coin** (log-loss 1.2 vs 0.69) and picks the actual winner only ~44% of decisive calls — **below chance**. Below-chance is not "no signal": it means the win% is **systematically inverted** (the policy backs the fast/offensive team; that team loses more — the Staraptor bias, quantified). Held-out Platt recalibration comes out with a **negative slope** and just edges the coin (0.6897 vs 0.6931) — real but *tiny*, because even **player-Elo $\approx$ coin (0.687)** here: Champions is near-unpredictable at the game level from sheets alone. **Consequences:** (1) the win% is a matchup heuristic, not a game predictor; (2) **DITTO was optimising a backwards signal** — building teams the biased engine loves (confirmed) — so its objective must be de-biased/flipped before "best team" means anything; (3) the durable value is the **validated damage** (exact against the Smogon damage calculator → CHOMP/ORB), which is genuinely not a coin.

Harness: `engine/backtest_winrate.js` , report `data/winrate-backtest.json` . **Policy validation (2026-07-23):** the behaviour-clone (the policy's backbone) predicts held-out human moves at **top-1 35.9% (CI 35.2–36.5)**, **top-3 71.6%**, cross-entropy 2.27 nats — beating the species-agnostic baseline (4.54) and uniform-over-moveset (2.91), so the priors carry real signal, but human move choice has genuine entropy (the clone is a *modest* predictor). A phase-conditioning improvement was tried and did **not** beat the proper score, so it wasn't shipped. This is a conservative lower bound on the full policy (the KO-take/Protect overrides only raise agreement on those turns). Harness: `engine/eval_policy.py` , report `data/policy-eval.json` . **So**

MEDICHAM's win rate is $P(\text{win} \mid \text{realistic cloned play})$, now with the clone's fidelity measured — not $P(\text{win})$ ground-truthed. Honest status: big improvement over the old 1v1 chain (which gave 0%/100%). Mirror 0.50, healthy spread, 400 rollouts in ~30ms, results carry a 95% CI on the site. **The damage math is now VALIDATED** against the Smogon damage calculator (MIT ground truth): with stats aligned, MEDICHAM matches the calc to the integer on 18/22 meta scenarios; after adding the Ruin quartet + Solar Power + Guts, it's **within 5% on 100% of scenarios, median error 0%** (worst 3% = 16-roll rounding). See `engine/validate_damage.js` and `data/damage-validation.json` . The remaining caveat is the *policy* (behaviour-cloned; over-credits speed control), not the damage numbers. **THERE IS ONLY ONE MEDICHAM NOW (2026-07-30)**. Will: *"LETS JUST CALL THE FUNCTIONAL MEDICHAM MEDICHAM, NO NEED FOR V3, FOLD OLD DEAD VERSIONS INTO A GRAVEYARD."* `engine/medicham.js` was the **v2** singles rollout — 1v1 in a doubles format, a hardcoded 14-move priority list, unseeded

`Math.random` , no team sheets, no megas. It is now `engine/graveyard/medicham-v2-singles.js` and **must not be read, fixed, or cited as MEDICHAM's state**; a session audited it by mistake and reported its limitations as ABRA's. Its two consumers were repointed at the doubles engine (`engine/ditto.js` , `tests/test-medicham.js` , the latter now guarding the live engine's three invariants — mirror symmetry 0.538, range, antisymmetry).

What MAG borrows from it: only `buildMon` and `dmgRange` . MAG never runs the rollout — so describing MEDICHAM as "MAG's damage calculator" is describing the borrowed half, not the model. **Code:**

`engine/medicham2-browser.js` ; `mcWinProb` / `mcWinProbI` in the site delegate to it (now Laplace-smoothed so a rollout can never read 0%/100%). Abilities patched from a curated meta map; move names + items from real ladder sets. **Open: Champions rule changes vs Gen 9 (sleep, paralysis, specific moves) are NOT**

yet modelled — pending the exact format rules (flagged rather than guessed). Also: DAgger/improve the rollout policy; Protosynthesis/Quark Drive stat boosts; Mega stat/type swaps (abilities are done, stats aren't). Validation harness: `engine/validate_damage.js`, report `data/damage-validation.json`.

Job: turn your seed team into the best version against the live meta. **Method (as of 2026-07-23):** (1) solves the **Nash equilibrium** over the archetype match-up matrix (`data/meta-nash.json`): Rain/Sand/FakeOut), (2) **best-responds to that equilibrium**, and — the key fix — the hill-climb now optimises the **grounded MEDICHAM value**, using JOLTEON only to shortlist candidates. Enforces the **item clause** (one item per team). (3) reports a per-archetype **matchup bar chart** ("how your team does vs the meta") and names your exploiters. **Two modes (as of 2026-07-23):** *Refine my team* (keep your core, only the highest-impact swaps that clear +5%) and *Build a perfect team* (full hill-climb). Both score against **all data-derived archetypes** (see below) with a weight floor so every threat counts, show an all-archetype matchup bar chart, and use accuracy-weighted, recoil-aware MEDICHAM as the objective. **Honest status:** optimises the **now-validated** MEDICHAM damage engine (was JOLTEON≈coin, which produced junk). The remaining caveat is the rollout *policy*, not the damage. Win-rate bars are Laplace-smoothed (never 0%/100%). **Code:** `runDitto(mode)` in `web/index.html`; archetypes from `engine/archetypes.py`; equilibrium math `engine/slowking/nash.py`.

STATUS 2026-07-30 — PARKED, WITH A NAMED FIRST STEP. Will: *"I WOULD LIKE TO IMPLEMENT DITTO AT SOME POINT."* Nothing currently `require s engine/ditto.js`, which reads as dead; it is not. It is a goal with known defects, and it was rebuilt-in-part this session:

Fixed. Its referee was v2's `winProb` — a **1v1 sequential-singles** rollout being used to judge four-Pokemon doubles teams. The "grounded rollout" meant to catch JOLTEON Goodharting itself had no spread damage, no redirection, no Protect and no positioning. Repointed at `winProb2`, the doubles engine; verified running (`medichamRank` returns 0.517 rather than null).

Still broken, and the rebuild Will approved:

1. **The objective is ~94% "add up six numbers."** It hill-climbs JOLTEON, which is additive (see above), so it converges on the top-6 by weight. **No synergy is representable** — no weather core, no redirection-plus-setup, no Trick Room pairing.
2. **The screen decides what the referee ever sees.** The coarse-to-fine design is sound — JOLTEON screens thousands, MEDICHAM re-ranks finalists — but MEDICHAM only evaluates what JOLTEON proposed. Fixing the referee buys little while the screen cannot *propose* a synergy team.
3. `coverage()` **does not measure coverage.** It reports "win rate vs teams that run Basculegion", but that is the same additive score filtered by which teams contain it. It cannot detect whether you have an **answer** — only how strong the teams that run it happen to be. The threat penalty is built on top of that.
4. **Unseeded** `Math.random()` in `loadMeta`'s shuffle, so the gauntlet and therefore the chosen team differ every run.
5. **It scores the six, not the four you bring.** `mean(six.map(spd))` averages all six; VGC is the bring.

The planned fix: pairwise terms over **roles**, not species — 258 species is ~33,000 pairs, far too many to fit on ~6,000 games, while `roles.py` tags 344 species into 52 roles. Same shape as DODUO's pair block, and subject to the same warning: fitted for resemblance it will lose. **Prerequisite done 2026-07-30:** the role vocabulary could not see megas at all, which is fatal here because the weather and terrain cores *are* megas. See ROLES.

Job: coach a real replay — take you to the turns that mattered. **Method (as of 2026-07-23):** parses the replay log to find decisive turns (KOs, losses), then runs a **clean move-by-move walkthrough** — big prev/next arrows, sprites + HP each key turn, and a bold **"what you should've done"** panel with the prescriptive fix. No Showdown iframe (dropped as clutter). **Works offline (file://):** coaches from a

locally-bundled set of recent games (`data/kad-replays.js`), with a recent-games picker and a raw-log paste fallback. **PORY win% wired in (2026-07-23) — RETRACTED 2026-07-25. Coefficients corrected and the tie MEASURED, 2026-08-04.** PORY is not a validated value net: its fitted weights reduce to `sigmoid(0.9809*alive_diff + 1.4093*hp_diff)` (restamped 2026-08-05, 5,883-game corpus; was 0.9943/1.4080 at 4,623), it ties that two-feature material baseline, and `turn` is structurally pinned to zero. The chip still renders, but it reports material arithmetic, not a learned value.

The coefficients quoted here were 1.256 / 1.544 for ten days and belonged to a different run.

Recomputed from the `weights` and `feat_std` actually stored in `data/pory-eval.json`, the reduction is **0.9943 / 1.4080**. 1.256 / 1.544 is commit `44e0fb0` (2026-07-24, `n_games` 7,381) — the run the retraction was written against, and the last one fitted on the **unfiltered store, bot games included**. `7f74236` (2026-07-26) put every model behind the clean filter and the coefficients moved to 1.0259 / 1.4347, then 0.9946, 0.9962, 0.9943 as the corpus grew. The doc was never restamped, so the retraction has been citing bot-contaminated coefficients as its evidence. `data/pory-eval.json` now carries a `reduced_form` block derived from its own weights, plus the per-run history, so this cannot drift again. **The retraction's substance is unaffected, and it is structural rather than numeric.** Every state is emitted from both perspectives with the label flipped, so the gradient on any column identical across the two rows cancels exactly: the intercept and `turn/10` are pinned to `0.000000000` at every generation, not shrunk to it. `my_alive` and `foe_alive` swap across the two rows and come back exactly antisymmetric (`+0.28071 / -0.28071`, summing to `0.000000000`), so they fold into `alive_diff`. Five features, two degrees of freedom, and more data cannot change that. **"Ties exactly" is now a measurement rather than an inference.** `engine/pory.py` was replayed on the identical 4,623-game sample and returned this file's weights, `feat_std` and log-loss bit-for-bit. Against a logistic on [`alive_diff`, `hp_diff`] alone — same gradient descent, same standardisation, same temporal split — PORY scores **0.629799 to 0.629778**, a paired difference of **+0.000021 (PORY worse), 95% CI [-0.000013, +0.000056]** clustered by game over 925 held-out games. The interval contains zero. Re-fitting on the current corpus (5,456 games) gives **-0.000001, CI [-0.000031, +0.000030]** — the tie holds at the larger sample. Restamped again 2026-08-05 at 5,883 games (the \$5f corpus-drift refit): baseline 0.623623, paired tie **+0.000001, CI [-0.000026, +0.000029]** over 1,177 held-out games. Three corpora, one conclusion; the retraction stands. `data/pory-eval.json` **said the opposite until 2026-08-04, and not because it was stale.** `engine/pory.py` gated its verdict on `hi < coin` and `hi < material_heuristic`, where `material_heuristic` is a crude 0.75/0.25/0.5 sign rule. PORY's interval genuinely clears that, so every re-run re-asserted "a real, calibrated value net" ten days after the retraction. Restamping the artifact alone would have been undone by the next run; the gate now reads the paired difference against the two-feature baseline, and the withdrawn string travels with the file under `withdrawn_verdict`. **Still to fix elsewhere:** `web/stadium.html:342` repeats 1.256 / 1.544 in prose. WEB's file — flagged, not edited. Original entry follows. Each key moment shows a **"you're at X%" chip** from PORY (`poryWin()` reads `data/pory.js`; `features = mons alive out of 4 + mean active HP + turn`). This is a validated per-turn readout (PORY: log-loss 0.567 vs coin 0.693, calibrated), unlike the still-heuristic prose fix.

Honest status: working offline; the prescriptive text is heuristic ("you traded X for nothing — Protect/pivot keeps it alive"), not yet equilibrium-grade — that's ALAKAZAM, later — but the win% chip beside it is real.

Code: `runKadabra`, `kadCoach`, `kadBuild`, `renderKad`, `poryWin` in `web/index.html`; bundle from `engine/refresh-site-data.py`. **Open:** deeper per-turn analysis (win-prob delta per decision) once the engine + value net are wired.

Job: answer "does archetype A actually beat archetype B" from real outcomes, so that nothing downstream has to assume a matchup. It is the input SLOWKING solves and the object the site's GURU booth renders.

Method: `engine/guru.py` labels each clean ladder game's two teams with one of 12 archetypes, tallies head-

to-head outcomes into a $12 \times 12 =$ **144-cell matrix**, and puts a **Wilson 95% interval** on every cell. `build/build_guru_js.js` projects that into `data/guru.js` for the browser. No simulation is involved and no model is fitted — it is a census with intervals. **Corpus: 5,265 clean games**, generated **2026-07-31 16:43**, and **26.1% behind** the 7,123 clean ladder games available on 2026-08-04. **Deliberately not regenerated.** It moves every number the GURU booth renders, so it is a joint pass with WEB, not a refresh. (It read 24.2% behind earlier the same evening; the figure moved while nothing about the artifact did, which is the treadmill `engine/provenance.js` now annotates with an absolute-power line — the 1,858 missing games can move a proportion by at most **0.61 points**, against a smallest measured split-half floor of 0.43.)

THE HEADLINE IS A NULL, AND IT HAS TO BE SAID IN THAT ORDER. The matrix finds **6 directed = 3 distinct** matchups whose 95% interval excludes 50%, taken one at a time:

matchup	p(row beats column)	95% CI	n
Charizard-Garchomp vs Trick Room	0.660	[0.526, 0.773]	53
Kingambit-Sneasler vs Sableye-Aerodactyl	0.648	[0.546, 0.739]	91
Gengar-Incineroar vs Sableye-Aerodactyl	0.621	[0.501, 0.729]	66

ZERO of them survives a correction for the family. There are $C(12,2) = 66$ unordered pairs, each its own 95% test, so **3.3 are expected to clear the bar with no real effect anywhere in the matrix** — and **3 appear**. The smallest exact two-sided binomial p-value in the whole matrix is **6.1e-3** against a Benjamini-Hochberg threshold of **7.6e-4: 0 survive FDR at $q=0.05$, 0 survive Bonferroni.**

`data/guru.js` publishes `n_decisive` (6), `n_decisive_corrected` (0) and the arithmetic under `multiplicity`, so the correction travels with the number instead of beside it. **What GURU establishes is that the matrix is descriptive structure. It does not establish that any archetype beats any other.**

Predictive test: `log_loss_matchup_prior` **0.7124** over 1,053 held-out games, against a coin's **0.6931** and a usage prior's **0.6928**; winner-pick accuracy **0.4982**. GURU is **worse than a coin** at predicting a single game. The artifact's own note says per-game prediction is expected near the coin because the format's ceiling is there — that is fair, and it does not turn 0.7124 into a pass.

*The competing 0.735 was traced rather than assumed, and it was not the site. 0.735 appears nowhere in `web/` or `app/` — the only hits are Golurk's damage and speed percentiles in the JOLTEON roster. It lives in `docs/SUMMARY.md`, attached to a run over **1,124 clean games and 11 archetypes**, and in `docs/THESIS-DEFENCE-REVIEW-2026-07-31.md` quoting it. `SUMMARY.md` is corrected to 0.7124 in this pass; the review document is dated history and is left alone. Both readings agree on the conclusion — GURU is worse than a coin at picking a single game — so the defect was a stale figure, not a changed verdict.*

The `n_decisive` bug, which is the reason this entry exists. `build/build_guru_js.js` reads `g.decisive`. `engine/guru.py` writes the list as `decisive_matchups`. The missing key gave `[]`, the generator then recomputed `n_decisive` **from its own empty fallback**, and shipped a provenance note asserting "ZERO statistically-decisive matchups on this population" as though it were a finding. The 144-cell matrix was byte-identical throughout, so nothing looked wrong. `venusaurmega` / `venusaur-mega`, in a new pair of files.

It was accidentally right, and that is worse than being wrong. The true corrected count really is zero — but it arrived there by a key typo, not by the multiplicity arithmetic above, and a conclusion produced by a bug **cannot be checked**. Anyone who verified it would have found the right answer and concluded the pipeline worked. A wrong number gets caught; a right number from a broken path does not, and it licenses the broken path. Three derived guards stop it recurring: every source key must be projected or named in `DELIBERATELY_UNUSED` with a reason; the source must agree with itself (`decisive_matchups.length === min(n_decisive, 20)`); and `build_guru_js.js --check` rebuilds the bundle in memory and diffs it, run by `tests/test-guru-derived.js` on every suite run.

Filed, not fixed (WEB's files): web/index.html:1845 gates a panel headed "These are the matchups we can actually trust" on GURU.decisive.length , which is the **uncorrected** 6, so it will render three matchups that do not survive multiplicity. It should read decisive_corrected . The same applies to isSig() in the matrix render and to the "statistically significant loop" claim, independently of that fix. **Code:** engine/guru.py → data/guru-matchups.json ; build/build_guru_js.js → data/guru.js ; tests/test-guru-derived.js . Consumed by engine/slowking_preview.py (its default MATRIX_FILE).

A note on this ledger, recorded because it is the same class of defect. docs/MODELS.md was found drifted on 2026-08-04 in three separate places and all three are corrected in this pass: MAG's fit read 6,091 games / 146,910 decisions / 53 features against data/policy-weights.json 's **8,414 / 220,613 / 58**; MAG's corpus line read 198,157 decisions from 7,507 games against the same file's **220,613 kept of 228,084 seen from 8,414**; and SLOWKING's headline mixture, exploitability, cycle and CI existed in no file on disk at all. A fourth reported drift did not reproduce: the mechanics census reads **102 live of 144 probed, 42 missing** in data/mechanics-census.json , docs/ENGINE.md:15 already prints exactly that, and docs/MODELS.md carries no census figure to correct — so 42/54 was not found here and nothing was changed for it. Checking the claim was cheaper than acting on it.

Job: the endgame — tell you the equilibrium-best move (and win %) on a live position. **Method:** the poker-AI stack (CFR → DeepStack → Libratus → ReBeL) adapted to VGC. engine/slowking/ : nash.py (equilibrium, verified on RPS/2×2), belief.py (public-belief-state + Bayesian filter), ismcts.py (simultaneous-move regret matching, recovers exact Nash), game.py (engine interface), solver.py (team-preview Nash + continual re-solve → bring mix + win%), value.py (learned leaf evaluator). **Preview-Nash built + evaluated:** engine/slowking_preview.py solves GURU's archetype matchup matrix to an equilibrium mixed strategy and grades it by **exploitability** (the spec's bar). Result → data/slowking-eval.json (+ data/slowking.js).

EVERY NUMBER THAT USED TO BE IN THIS PARAGRAPH EXISTS IN NO FILE ON DISK. Corrected 2026-08-04, quoting the artifact. It read: equilibrium Kingambit-Basculegion 0.84 / Garchomp-Incineroar 0.16, exploitability Nash ≈ 0 vs uniform 0.109, a named non-transitive cycle Charizard-Venusaur → Whimsicott-Garchomp → Garchomp-Incineroar at ~0.10 edge each leg, and a greedy-vs-Nash gap CI upper bound of 0.27. None of the four is in data/slowking-eval.json , in data/slowking.js , or anywhere else in data/ . They are a 2026-07-23 run whose artifact was overwritten, and the prose outlived it — which is the same failure as the fourteen HANDOFF-*.md files, inside the living ledger that was supposed to replace them.

What the artifact actually says (data/slowking-eval.json , written 2026-08-03 15:15 over data/guru-matchups.json 's 12 archetypes and 5,265 games):

quantity	value
equilibrium mixture	Gengar-Incineroar 0.6602 / Charizard-Garchomp 0.2182 / Pelipper-Archaludon 0.1210
exploitability	Nash 0.0001 , greedy single deck 0.0410 , uniform 0.0761
greedy – Nash	0.0409 , 95% CI [-0.0001 , 0.1735] — the lower bound does not clear zero
strongest cycle	Charizard-Garchomp → Kingambit-Garchomp → Incineroar-Whimsicott, min edge 0.095
is that cycle supported?	supported: false — legs rest on 49, 37 and 15 games, none clears 50%, and it is the strongest of 1,320 candidate triples

So the honest reading is the opposite of the one the old paragraph gave: mixing is **not** shown to beat greedy here (the gap CI includes zero), and the cycle is what the best of 1,320 searched triples looks like when there is no structure — LESSONS §10, in the model that lesson is named after. CI propagates matchup-count uncertainty by Beta resampling. Test: tests/test-slowking.py (RPS hand-check + shipped-artifact invariants), gated in CI.

AND `data/slowking-playstyle.js` **IS NOT A PLAYSTYLE RESULT. Found 2026-08-04; not repaired here.** `engine/slowking_preview.py` takes the matrix from `MATRIX_FILE`, which defaults to `data/guru-matchups.json`, and takes only the OUTPUT FILENAME from `TAG`. Run with `TAG=playstyle` and `MATRIX_FILE` unset, it writes a GURU run under the playstyle name. That is what is on disk: `data/slowking-playstyle.js` has a payload **byte-identical** to `data/slowking.js`, and `data/slowking-playstyle-eval.json` is a **byte-identical file** to `data/slowking-eval.json` — 5,265 games, 12 species-pair archetypes, 1,320 triples. The real playstyle matrix (`data/playstyle-matchups.json`) holds **2,860 games over 8 playstyles**, and re-running against it reproduces the figures **this ledger already publishes** in the MACHAMP entry above — 336 candidate triples, a leg resting on 5 games, greedy-Nash **0.026 CI [-0.0001, 0.1498]**, mixture Rain 0.81 / Setup 0.17 / FakeOutBalance 0.03, and the verdict "no material exploitability gap... this meta is close to transitive at this granularity". **The docs are right and the artifact is wrong**, which is the rare direction. The repair is one command with both variables set; it moves every figure the site's cycle panel renders (`app/index.html:907-923` reads `window.SLOWKING_PLAYSTYLE`), so it is a WEB pass, not a refresh. The generator should refuse to write a `TAG`-named file from the default matrix.

Honest status: preview-Nash is now a **real, tested model on real data** (exploitability + baseline + CI), not just a chassis. The in-battle search (IS-MCTS/PIMC → ReBeL) is still a rung below target and not yet wired to the engine — that's ALAKAZAM. **Code:** `engine/slowking_preview.py`, `engine/slowking/*`; white paper `docs/POKER-TO-POKEMON.md`. **Open:** wire `ChampionsGame` to the real engine; PIMC → outcome-sampling MCCFR / PBS re-solving; train the value net via self-play.

value net: `engine/train_value.py` reconstructs per-turn HP state and regresses the outcome → `data/value-net.json`. Held-out **log-loss 0.6536**, Brier **0.2306**, accuracy **61.4%** against a coin's 0.6931 and an alive-count heuristic's 0.662, on 10,125 games / 15,544 test states. Calibrated in the middle and compressed at the tails, where the data is thin ([0.8,1.0]: predicts 0.85, observes 0.80 on n=225). It's SLOWKING's leaf evaluator. (The `0.682` this line carried until 2026-08-04 was in no artifact; the committed file said 0.6638.)

THE WALK WAS DISCARDING EVERY FORME-CHANGED BODY, SILENTLY. Fixed 2026-08-04. `idn()` normalises punctuation and nothing else, so the event stream's `charizardmegay` never matched the bring list's `charizard`, `side_of` returned None and the event was thrown away. Measured on 4,000 clean games: **21.7% of faints, 22.7% of damaging events and 20.8% of all damage**, with at least one discard in **96.5%** of games and **97.6%** of the discarded targets megas. The visible symptom was that **88.9% of clean games ENDED with both sides still holding bodies** — the value net was being trained on trajectories in which almost nobody ever loses their team. It is `venusaurmega / venusaur-mega` again, in a file that had no lookup at all; the fix routes both sides of the comparison through `engine/mc_key.js`, which gained the verb it was missing (`mcKey.base / mcKey.bases`) rather than growing a fourth hand-rolled resolver. **What it moved, paired on identical test states, 1,445 held-out games / 10,120 states:** log-loss **0.6634 → 0.6520**, paired difference **-0.0114, 95% CI [-0.0183, -0.0041]** clustered by game; accuracy **59.72% → 61.47%**, paired **+1.75 pts, CI [0.50, 2.94]**. The mechanism is legible in the weights — `hpDiff` moved **0.169 → 0.377**, because a fifth of all damage had never been applied and the feature was attenuated toward zero. **Say the size honestly: the pairing is what buys this, and it is small.** Twenty split-half cuts of the fixed arm alone spread by a **median 1.87 points** of accuracy (range 0.30–5.37), so the +1.75 effect is **INSIDE** the floor for an unpaired comparison — two runs on different samples could not tell these value nets apart. And the destination is unchanged: 61.4% is still well under the **66.92%** in-sample ceiling for this feature class and under the live leaf's **67.97%**. A correctness fix worth making on its own terms; not a capability change. **Residual, measured not assumed: 1.7% of damaging events are still dropped**, and 1,613 of the 1,625 are one species. `MC.mons` carries `floette-mega` with `base: "floette"`,

the store's bring lists hold `floetteeternal`, and there is no `floette` row — so the chain does not close. The rest are in-battle formes the mega table does not cover (`mimikyubusted` 274, `morpekohangry` 48, `castformsnowy` / `castformrainy` 11). Both are dex-data gaps and are filed, not patched here: reaching for the Showdown dex to close them would make `train_value.py` produce different numbers depending on whether `SHOWDOWN_PATH` is set, which is this project's named failure in a new place.

self-play: `sim/generate-dataset.js` writes engine games into the store schema (unlimited, unbiased data).

flywheel: `engine/flywheel.py` — self-play → retrain → re-evaluate → report the delta. The thing that makes ABRA learn over time. **live data + auto-refresh (2026-07-23):** `engine/durable-ingest.js` pulls new ladder replays; a **daily scheduled task** runs it, then `engine/refresh-site-data.py` regenerates `data/archetypes.json` (via `engine/archetypes.py` — archetypes discovered from the games by k-means, not hand-listed), `data/live.js` (counts + archetypes the site loads live) and `data/kad-replays.js` (offline KADABRA bundle). So the site's numbers and meta **grow on their own**.

CHOMP — the bring-4/lead-2 decision engine (Showdown userscript). Picks your best 4 and 2 leads by exact damage over the opponent's whole six. *Open:* bring/lead should be a minimax matrix game (`nash.py`), not greedy coverage. **CHOMP-EV proof (2026-07-23) — honest NULL.** The winnable test (do CHOMP's brings beat humans' actual brings on held-out games?) was run over 1,205 human games

(`engine/chomp_ev.js` → `data/chomp-ev.json`). CHOMP's damage-coverage bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), **ties** an Elo and a usage-prior baseline, and winners

are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). Robust to dropping all forfeits; a measured selection audit shows the mild eval-set bias *favours* CHOMP, so the null is conservative.

The bring decision sits at the same near-coin ceiling as pre-game prediction — CHOMP's damage math stays validated and useful as a calculator/EV display, but "CHOMP builds better brings" is not yet empirically supported. This is the guardrail that stops a DITTO-style on a signal-free metric. **Belief-**

weighting tried (2026-07-23): scoring coverage vs the opponent's *likely 4* (top-bringRate mons) instead of all 6 also ties the coin (log-loss 0.6924 vs 0.6931), so belief-awareness alone doesn't rescue it — the bring decision is at the format ceiling, full stop. *Next (if pursued):* full XATU-set belief + SLOWKING lead stage-

game + PORY leaf value, then re-run this exact harness and measure the lift. **ORB** — On-battle Read Board, the damage calculator. **Decision (2026-07-23):** rather than fork the Smogon calc (the hosted one can't be auto-filled cross-origin, and a fork needs a build), ORB is a **validated Smogon-grade substitute built into the CHOMP dock** — same damage engine validated against the Smogon damage calculator, reading the **live battle:** real stats/items, boosts on both actives (Intimidate/setup), weather, terrain, Helping Hand, spread, screens; it prints the conditions it applied. One-click install, auto-updates. (`docs/ORB-smogon-fork.md` kept as the record of the fork option we chose against.)

-
- `engine/eval_harness.py` — held-out log-loss / Brier / calibration vs coin, Elo, usage baselines with bootstrap CIs. **The bar every model must clear.**
 - `engine/calibrate.py` — temperature scaling.
 - `docs/THESIS-REVIEW.md` / `THESIS-REVIEW-v2.md` — strict self-critique with fixes (willing to scrap/rebuild).
 - `docs/COMPETITORS.md` — VGC-Bench, PokéLLMon, offline-RL transformers, and how we refine them.
 - **~~Non-transitivity:** `data/nontransitivity.json` — the meta is rock-paper-scissors.~~ **WITHDRAWN 2026-07-26.** The file was computed 2026-07-23, two days before the quality filter, and nothing regenerated it — so the cycles it showed were measured over a corpus that is 87% bots, forfeits and stubs. Re-run on clean data, SLOWKING's equilibrium collapses to **100% on a single option** with **zero** gap between mixing and picking the best, and the clean GURU matrix contains **0 decisive matchups**. The file is deleted rather than kept, because a stale artifact on disk is how a retracted claim gets quoted again.

The honest reading is *no usable input*, not *mixing does not help*: a Nash solution over a matrix of noise says nothing either way. See CHANGELOG 3.16.0.

DONE (2026-07-23): the engine's damage math is validated against the Smogon damage calculator — within 5% on 100% of 31 tested scenarios (`engine/validate_damage.js` , `data/damage-validation.json`). MEDICHAM/DITTO no longer rest on unverified numbers. **Next priorities, in order:** (1) get the **Champions rule changes** (sleep, paralysis, moves) from the format and model them — the current biggest data gap; (2) harden the rollout **policy** (the last GIGO lever); (3) grow the dataset via the daily pull + self-play so the discovered archetypes and win rates sharpen.

Job: describe every team by the set of functional roles it reveals, instead of forcing one archetype label.

Method: 26 roles (speed control, weather, terrain, disruption, status/debuff, priority, prankster, setup, healing, screens, walls, pivot, trapping, perish, ally-support, item-disruption, physical/special attacker). Each **species earns a role from data** — credited once it is observed doing it (≥ 2 times). Multi-effect moves carry several *factual* roles (Matcha Gotcha = attack+heal+status; Body Press = wall+attack; Knock Off = attack+item-strip; Fake Out = tempo, not attacker). Role *presence* is binary and data-justified; graded strength is **not** hand-set — it is the learned output of the NMF (see below). Team role vectors are built from the **team-preview six** (leak-free). Outputs a **role-pair matchup matrix** with Wilson CIs. **Honest status / result:** the role-pair matrix **pools the data** — median cell **n=20** across 1,051 cells (2026-07-25, clean games) vs the old single-label archetype cells of $n=11-18$. Pooling still wins, but only just — and the 7,971 published in v2.6.0 was retracted in 2.7.0 as an over-tagging artifact. But predicting the winner from **preview roles ties a coin** (held-out log-loss 0.694 vs 0.693) — consistent with the sheet-level null. The value is descriptive + attribution, not prediction. Per-role logistic coefficients give **win-credit per role**; KO-credit per species comes from the turn log. **CAPABILITY LAYER, added 2026-07-30 — because team preview never shows a mega.** Will: *"ROLES ARTIFACTS IS ALSO VERY OLD"* / *"WE HAVE SMOGON DATA."* The artifact held 280 tagged species and **zero mega formes**, while megas are 26.0% of this format's usage. The failure split cleanly: `tyranitar` `weather_sand` 95.2%, `pelipper` `weather_rain` 97.9% and `torkoal` `weather_sun` 94.4% all worked, because those abilities sit on the base forme; `raichu` had no `terrain_electric` despite Raichu-Mega-X being the format's only Electric Surge, and `charizard` and `swampert` were **absent entirely**.

It was not a bug in this file. Checked against the store directly: of **58,920** team-preview species names in clean games, **zero** are mega formes (the only `/mega/` match is Meganium). That is correct Pokémon behaviour — preview shows the base and the mega is revealed only on evolution — so no amount of regenerating could produce them.

Fix: a `capability` block derived from `data/smogon-priors.json` , which carries mega formes as first-class rows *with* their abilities, over the whole ladder instead of a few thousand replays. Mega rows are folded onto the BASE name, read from `mega-dex-official.json` 's `base_species` rather than by stripping the name, because teams are built from base names.

Kept separate on purpose. `roles` remains $p(\text{role} \mid \text{species appears})$, measured with Wilson bounds; `capability` is a **reachability set** — can this species play this role at all. Merging a presence flag into a measured distribution would have quietly corrupted it. Presence, not rate, is load-bearing here: Smogon records the mega rows' ability slot unreliably (Raichu-Mega-X reads "No Ability 81%, Electric Surge 19%"), so a frequency filter would discard the exact capability sought.

`charizard` -> `weather_sun`, `abuser_sun` from `charizardmegay` (Drought + Weather Ball + Solar Beam) `raichu` -> `terrain_electric` from `raichumegax` `swampert` -> `abuser_rain` from `swampertmega`

Noise checked rather than assumed: 17 species reach `weather_sun`, which looked loose until verified — Whimsicott runs Sunny Day at **16.2%**, Liepard 12.3%, Sableye 6.8%, Klefki 5.2% (Prankster sun is a real archetype), while Torkoal and Charizard-Mega-Y show *no* Sunny Day because they set it by ability. Both routes captured, neither invented.

Regenerated on 4,910 clean games (was 2,653), **344 species tagged** (was 280), 200 with a capability set, 1,101 matchup cells, median $n=69$. **The null holds:** held-out log-loss `roles=0.6933` against a coin's `0.6931`, CI (0.6880, 0.6981), accuracy 0.508 — role-level winner prediction still ties a coin on $1.85\times$ the data. The capability layer is team-building **vocabulary**, not prediction, and does not change that.

Known downstream staleness: `xatu-context-sets.json` and CHOMP-EV are built on this artifact and `roles` moved with the larger corpus. Regenerate in dependency order (`roles` → `context` → CHOMP-EV) before quoting either. **Code:** `engine/roles.py` → `data/pokemon-roles.json`, `data/role-matchups.json`, `data/roles-eval.json`. Tests: `tests/test-roles.py` (19).

Job: how many wins a species adds over a freely-available replacement, controlling for teammates and opponents. **Method:** ridge-regularized **Adjusted Plus-Minus** (basketball RAPM) — logistic regression of game outcome on the difference of team-preview species-indicator vectors. Replacement = 20th-percentile β ; $WAR = 0.25 \cdot (\beta - \beta_{\text{repl}}) \cdot \text{games}$ (the logistic wins conversion). Ridge shrinks rare species toward zero.

Honest status / result — WITHDRAWN 2026-07-25. WAR does not beat a coin on clean data.

The previous entry read: "the species model **beats a coin** (held-out log-loss $0.6875 < 0.6931$) and beats the rating baseline (0.6905) — so *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not." That was measured on the **unfiltered** store.

Run both ways on the same store (v3.2.0):

	held-out log-loss	vs coin 0.6931	accuracy
unfiltered, 8,356 games	0.6860	beats it	0.539
clean, 1,061 games	0.7048	worse than a coin	0.502

The mechanism is visible in the coefficients: Basculegion's WAR falls from **281.87 to 23.64**, and Basculegion is one of the six Pokemon four undetected bot accounts played in 1,446 identical games. The model was learning which species belonged to the highest-volume account, not which species win. Charizard — also on that team — is the largest negative in both runs, the same artifact inverted.

Current status: a null. Preview species composition sits at the same coin-flip ceiling as preview roles and raw sheets. WAR magnitudes are retained as a descriptive ordering only and must not be quoted as evidence that species choice predicts outcomes. **Code:** `engine/war.py` → `data/war.json`.

Job: discover roles and archetypes from the data instead of hand-declaring them. **Method:** Non-negative Matrix Factorization (Lee & Seung 1999; Label Distribution Learning framing, Geng 2016). Two cuts: (1) team×MOVE usage (usage-weighted, so the closed-sheet censoring skew is down-weighted) → offensive cores; (2) team×ROLE → **emergent archetypes**. A team is a non-negative *blend* of factors, never one hard label; a move's loading on a role is **learned, not typed** — this is where graded primary/secondary strength legitimately comes from. **Honest status / result:** role-level factorization is the clean cut — recon-err **0.53**, six interpretable archetypes (Intimidate+Fake-Out control; physical offense; special offense+sustain; **bulky wall+screens+redirection**; Tailwind+Encore; priority). Move-level is coarser (recon-err 0.79; attacking moves dominate). Rank and the human names are the only non-data choices; rigorous rank/weighting selection by **topic coherence** (Mimno 2011) is the noted next refinement — reconstruction error is not comparable across weightings. **Code:** `engine/nmf_roles.py` → `data/nmf-roles.json`, `data/nmf.js`.

Vocabulary census: `engine/vocab.py` → `data/vocab-usage.json` (tags every move/ability/item, counts real battle usage; curated roles cover 90.4% of non-neutral usage). Site booth: the **Role Foundry** (Smeargle) in `web/index.html`.

Job: measure whether players spend spare move slots answering the metagame, rather than on their own gameplan. **Method:** cross-sectional, by necessity — the store spans 3 days, so the natural temporal test ("does Fighting usage rise AFTER Kingambit rises?") has no identifying variation and was not run. Instead: for each species, compare the meta-weighted type coverage of its RARE moves (a tech slot, $\leq 12\%$ of its sets) against its STANDARD kit ($\geq 30\%$), where coverage weights the real 18×18 type chart by each threat's current prevalence. Paired within species, bootstrap CI over species. **Result:** tech slots carry **+0.0386** more meta-weighted coverage than standard kit, **95% CI [0.0155, 0.0617]** — **excludes zero**, positive in 90/148 species. Concretely, vs **Kingambit** (Dark/Steel, 29% of teams) the top tech answers are Incineroar's Close Combat (Fighting, **4×**, 127 uses) and Blastoise's Aura Sphere (4×, 90) — the "rogue Fighting coverage for Gambit" pattern, measured. **Code:** `engine/counterplay.py` → `data/counterplay.json`.

Job: give the damage engine real mega stats, types and abilities. **The gap:** the engine dex held ONE mega forme while Charizard-Mega-Y alone appears in ~906 sets, so every mega calculation silently used base-form stats. Separately, the ingest never parsed mega evolution, leaving 904/906 Charizard-Mega-Y sets with a blank ability. **Source:** Showdown's own `pokedex.json`, the data the server runs this format on. Champions invents megas that do not exist in mainline (Raichu-Mega-X/Y, Glimmora-Mega), so memory or a canonical dex is not a valid source. **Honest limit:** a mega's ability can NEVER be read from replay logs — mega evolution announces nothing. Log harvesting is retained only to discover which formes exist. Level-50 stats use an assumed competitive spread and are labelled as an approximation, since closed sheets never reveal real EVs. **Result:** 67 mega formes in the engine dex; damage validation unchanged at 100% within 5%. **Code:** `engine/mega_harvest.js` (discovery), `engine/build_mega_dex.js` (official), `engine/merge_mega_into_engine.js` (merge).

Job: detect when a Pokémon on screen is actually Zoroark wearing its name. **Method:** Illusion copies the name, not the moveset. If the apparent species cannot legally learn a move and Zoroark can, the disguise is *proven* — a legality contradiction, not a probability. Learnsets from Showdown, walked through prevo/base chains; species with no learnset data are skipped rather than guessed at. **Result:** on 395 Zoroark team-sides, **156 proven disguises** (0.39 per side). Most common disguise Whimsicott (16); the moves that give it away are Hyper Voice (32) and **Bitter Malice (30)**. A floor, not an estimate — a Zoroark that only clicks shared coverage is invisible to this test. **Why it matters:** on ~1.4% of teams the Pokémon you are planning against may not be that Pokémon, and the reveal is a large sudden information gain. This is XATU's problem, not a static dex's. **Code:** `engine/illusion.js` → `data/illusion.json`.

Job: replace a fixed usage table with a per-slot *information state* that narrows as the opponent proves things. **The distinction:** "Kingambit usually runs Sucker Punch" is a statement about the population, and it never changes during a game. But in a closed-sheet format nothing is known until it is proven, and every reveal is information. The cheapest and hardest constraint is one a usage table cannot express: **a Pokémon has exactly four moves**. Once four are revealed the set is closed, and every other move in the usage table — however popular — is impossible. There the prior is not merely imprecise, it is wrong. **Method:** prior $P(\text{move} \mid \text{species})$ fitted on TRAIN games only; on held-out games we walk the turns in order and predict each move before seeing it, using only what that Pokémon revealed earlier in that same game. Scored by cross-entropy against two baselines (uniform, and the usage prior). The "already revealed" boost is **measured on train data**, not chosen: $P(\text{next move is one already seen}) = 0.558$, and the boost is its odds form, 1.26. An earlier draft asserted 3.0, which flattered the model — the measured value is smaller and so is the gain. **Result:** cross-entropy **1.824 vs the usage prior's 1.863** (uniform floor 6.06), top-1 **35.7% vs 34.8%**. Improvement **0.028, 95% CI [0.024, 0.031]** clustered by game — clears zero. On the 2.4% of events where all four moves are already known, belief **1.695 vs prior 2.219**: that is the four-move cap doing the work. **Honest scope:** this is the move slot only. Items and abilities are unknown-until-proven in the same way and are tracked as possibility sets but not yet scored. **EVs are different in kind** — a damage roll bounds an attacking stat to an interval and moving first proves only an inequality, so an EV spread never collapses to a

value the way an ability or item does. That needs a separate interval estimator. **Code:** `engine/xatu_belief.py` → `data/xatu-belief.json`. Tests: `tests/test-xatu-belief.py` (14, incl. the uniform floor derived by hand as $\ln(V)$ and the boost re-derived from its own probability).

Job: predict an opponent's set before a single turn is played, using the only information that exists at preview — the other five Pokémon. **Why it was needed:** the belief-state tracker only sharpens once something is revealed, but CHOMP decides at turn zero. That looked like a dead end for feeding better beliefs into the bring decision, until the obvious point: a set is chosen to fit a *team*. Pelipper on the roster makes Swift Swim plausible; no rain setter makes it close to pointless. **Method:** $P(\text{move} \mid \text{species, teammate features})$ where the features are public at preview — rain/sun/sand/snow setter, Trick Room, Tailwind, redirection. Each (species, context) cell is shrunk toward the species prior by $n/(n+K)$ with $K=12$, so a context seen once carries under 8% weight and cannot manufacture a signal. Fitted on train games; scored on each Pokémon's FIRST revealed move in held-out games; clustered by game. **Result:** cross-entropy **2.4415 vs the bare prior's 2.5257**, top-1 **43.0% vs 40.9%**, improvement **+0.084, 95% CI [0.074, 0.094]**. That is a *larger* gain than the in-game belief tracker (+0.028) — and unlike that one, it is available exactly where CHOMP needs it. **A domain claim, checked:** "Basculegion is Swift Swim on rain and Adaptability otherwise." Its first move, with a rain setter on the team vs without: Wave Crash **46% vs 27%**, Last Respects **25% vs 48%**, Flip Turn 19% vs 8%. Roughly a 2x swing in both directions — the ability split is visible in move choice without ever observing the ability. **Code:** `engine/xatu_context.py` → `data/xatu-context.json`. Tests: `tests/test-xatu-context.py` (14, incl. re-deriving the shrinkage weight by hand).

Job: remove the sample-size constraint for every question that is about the GAME rather than about PEOPLE. **Why:** 1,124 clean games can only detect an edge of ~4.2 accuracy points over a coin; a 2-point effect needs ~4,900, and human replays arrive at ~330 clean games/day. Every preview-level null in this project sits inside that blind spot. **Method:** plays the **official** Showdown Champions engine (pinned `20ad99f`) against itself on **real six-Pokémon teams sampled from the clean store** — 1,257 distinct teams, sampled by distinct team rather than by game so a bot's team contributes once. Unrevealed set slots are filled from Smogon's official moveset statistics. Battle logs go through the SAME `extract()` as a downloaded replay, so self-play records are identical in shape and every downstream reader works unchanged. **Honest status:** built and gated. 1,000 games, 0 discarded. `engine/validate_selfplay.js` runs 13 checks — store shape (S7), set realism, determinism, and mirror symmetry at **51.0%, 95% CI [45.4, 56.6]** on 300 battles, so the harness has no side bias. **What it cannot do:** self-play produces **zero** evidence about people — whether players tech for the metagame, whether rating predicts, what a human will bring. Those need real games. The current batch also uses the **random** policy, which is right for matchup structure and plumbing but must **not** train a value net: a model would learn "P(win) when both players move at random". The behaviour-cloned policy is the next step, and VGC-Bench's cross-evaluation found clone-then-self-play (BCSP) is what actually wins. **Two bad batches shipped before the gate existed**, and nothing detected either: one filled every unrevealed move slot with Tackle (13% of move events), one used a flat 11/11/11/11/11/11 spread that understated Garchomp's Attack by 13%. That is why the gate exists. **Code:** `engine/mew.js` → `data/games.selfplay.jsonl` (gitignored, seed-reproducible, every record stamped `source:"selfplay"`). Viewer `web/mew.html`. Paper `docs/MEW-whitepaper.md`.

Job: decide a move by looking at the other side of the field, instead of by how popular the move is. **Why:** the behaviour clone answers only *what does this species usually click?* Two gaps followed and no prior-tuning could close them — super-effective moves at 9.7% against a real 21.4%, moves that outright failed at 9.7% against 2.5%. It also made every `build_lab` number a measurement of what beats **bad** play. **Method:** three files. `engine/board.js` reconstructs the state a decision was made against and turns (move, target) pairs into **58 features** (12 at 3.21.0); `engine/fit_policy.js` fits those features to real human clicks by **conditional**

logit (McFadden 1974) over **8,414 clean open-sheet games and 220,613 decisions** (176,580 train / 44,033 held out); `engine/magnemite.js` plays the fitted distribution inside the official engine. `mew.js` -- policy score .

*Every figure in that line was corrected 2026-08-04 and none of them was a typo. It read 53 features / 6,091 games / 146,910 decisions / 117,824 train / 29,086 held out. The artifact — `data/policy-weights.json` , generated 2026-08-04T02:17:31Z — carries features of length **58** and corpus of **8,414 / 220,613 / 176,580 / 44,033**. The ledger was describing the fit before last, and a second heading two screens down still said "56 FEATURES AS OF 3.29.0" while the method line above it said 53, so the file disagreed with itself as well as with the artifact. This is the drift `docs/MODELS.md` 's own header warns about, recurring in the entry for the model at the centre of the refit. Quote `data/policy-weights.json` ; it is one `corpus` object and it cannot drift.*

REFITTED ON THE FOUR-CHANNEL SHEET, 3.40.0 (2026-08-04T23:37Z artifact). Will's decision: open team sheets always; closed sheets deferred. `data/policy-weights.json` now reads corpus **8,856 games / 231,722 decisions (185,560 train / 46,162 held out)**, `fitEnvironment.sheet_channels` = `[nature, item, ability, moves]` , `matches_player`: `true` , and a **point-of-use reach counter** — the declared ability/moves arrived on the board for 99.67% of scored decisions — so the environment match is measured, not asserted. The two-channel incumbent is preserved (`data/policy-weights-presheet.json`) and frozen in release `d3d04b669e18` as arm A of the pending paired held-out comparison against the 0.192-point noise floor (`engine/sheet_channel_value.js` , not yet run). **The JOINT (pair) layer is NOT yet refitted** — until it is, the pair layer prices against the two-channel board, and no improvement claim exists for either layer.

BOTH HALVES CLOSED, 3.41.0 (same night). The joint layer is refitted on the four-channel sheet (`data/policy-weights-joint.json` : 95,886 usable joint turns of 101,459; `fitEnvironment` counters 99.7%; held-out pair top-1 9.8% → 12.2% with the joint terms). And the channel VALUE is now a measurement (`data/sheet-channel-value.json` , 44,982 paired held-out decisions vs the frozen two-channel incumbent): the sheet buys **+0.005087 logL/decision end-to-end [0.003854, 0.006331]** — real, clears zero — and **no demonstrable top-1 gain** against a 0.331-point split-half noise floor. Say it exactly that way: MAG prices decisions better with the sheet; a click-rate improvement is not yet shown.

Two changes to how the policy is USED beat every change to what it knows. Measured 2026-07-30: taking the best move instead of sampling is worth **+12 points raw / 79.7% of decisive pairs**, and self-play policy improvement (`engine/train_policy.js` , REINFORCE with a trust region) wins **55.9%**. Over the same period **four separate feature additions produced four measured nulls**, and an overdispersion check across teams (~1.00, against 1.169 for a known real effect) says those nulls are genuine rather than a real effect hidden by team heterogeneity. **The objective is the binding constraint, not the knowledge** — which is why DODUO's next test is a retrain rather than more features.

Facts that reached one consumer and not the next (all fixed 2026-07-30). Every integrity bug found that day had one shape. Priority blocking sat in the tag artifact read by `clickFragility` alone, so **Sucker Punch beat a Farigraf in every rollout ever run**. The sheet's item and ability reached `switchIn` but not `switchFeatures` , the path that actually chooses the switch. A switch-in's own ability never reached the estimate at all: over 40,001 matchups, declaring `intimidate` , `drizzle` or `drought` moved the vector in **0** of them against a `levitate` control's 2,754 — so MAG weighed bringing Incineroar in against the foe's full Attack. Now 9,227 / 3,463 / 3,438. Modelling the drop alone would have been *worse than modelling neither* (Intimidate into Kingambit is +2 Attack for them), so the whole three-stage drop pipeline runs: Contrary inverts, Clear Body deletes, Inner Focus/Own Tempo/Scrappy block Intimidate by name, Guard Dog converts to +1, Mirror Armor reflects, Defiant and Competitive retaliate. All derived by calling the dex's own handlers against a recording stub — no ability or weather is named in `board.js` . **Nothing in it is asserted.**

Every "this move cannot work now" test reads a dex **data field** — `move.status` , `move.sideCondition` , `move.pseudoWeather` , `move.weather` , `move.stallingMove` — against tracked state. No move is named anywhere in `board.js` , so a new regulation needs no edit (S13). The weights are estimated, never typed, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because it stops being evidence the moment it becomes the objective. **Why open team sheets:** a choice model needs the **choice set**. A normal replay reveals only moves that were *used*, so alternatives reconstructed from revelation are biased by revelation itself. Open sheets publish all four moves of all six up front. **Fit, held out by GAME** (decisions inside a game are correlated, so splitting by decision leaks): logL/decision **-1.6006**, top-1 **33.6%** — against the behaviour clone alone at -1.9302 / 27.1% and uniform at -1.7627 / 24.1%. In-sample -1.5997, so it is not memorising; weights identical at 200/300/500 iterations. **Measured out of sample, 600 seed-matched battles per policy:** super-effective **9.71% → 14.91%** (real 21.37%), failed moves **9.68% → 6.34%** (real 2.47%), immune **4.30% → 2.92%** (real 1.91%), Protect-type **21.62% → 16.71%** (real 13.87%). Both target gaps roughly halved, and 427 of 591 games survive the quality filter against the old policy's 382. **Most of the win was aiming.** `RandomPlayerAI` chooses which foe to hit with `prng.random(2)` before `chooseMove` is called, so the target was a coin flip however good the move choice was — and in doubles aiming is most of what "super effective" means. **It samples, it does not take the best move** — a greedy bot sails past 23.4% super-effective and is *less* human. Same argument as DEFENSE §2. **Two findings worth keeping.** The behaviour clone alone is a *worse* probabilistic model of human choice than choosing uniformly at random, and the fit weights it at only +0.25 — it is far too confident about the popular move. And the largest learned effects are not damage terms at all, they are the "this move is already dead" terms at -2.3. Reading the board is mostly about **not clicking moves that cannot work.** **58 FEATURES as of data/policy-weights.json 2026-08-04 (this heading read 56 and was stale; the three below are the large ones added at 3.29.0).** `data/tags.json` derives 96 move tags with their parameters and `engine/tags.js` exists to load them; `board.js` read NONE of them, and 72 of the 96 reached no consumer at all. The symptom Will spotted: MAG scored **Tailwind and Protect identically at -1.54**, because the only things firing on a Tailwind click were `accuracy` , `isStatus` and `priorLogP` . There was no speed-control feature in the 53.

feature	fires when	weight
<code>speedSwing</code>	it flips speed order IN MY FAVOUR; zero when already faster	+0.983 [0.933, 1.032]
<code>screenValue</code>	it halves incoming damage AND something hits hard, graded by CATEGORY	+1.128 [1.031, 1.225]
<code>healValue</code>	it heals me AND I am hurt; zero at full HP	+2.220 [2.004, 2.436]

Written as CONDITIONS rather than flags, and that is why they fired where 3.28.0's four additions measured null: a bare "this is Tailwind" cannot help a one-ply scorer, because the payoff is on later turns. What one ply CAN see is whether the condition making it worth doing is true now.

CHOICE LOCK (3.29.0). `fit_policy.js` handed `candidates()` all four sheet moves with no legality filter, so a choice-locked human appeared to have ~9 options when they had 4 — a WRONG DENOMINATOR in the conditional logit, on 6.52% of items. Live play was never affected (the request marks the rest `disabled`). After the refit, six of eight switch features clear zero, and **switches now win the argmax:** greedy play went from 222 switch events per 60 games (all forced post-KO) to 239, where before the refit `--switching` changed nothing at all.

THE OPPONENT MODEL — job 2 of ALAKAZAM, off by default (3.29.0). `incomingThreat` took a MAX, the foe's hardest available hit, and nine features are built on it. Measured: the foe's lead clicks a damaging move **52.9%** of the time and MAG assumed 100% AND assumed it was the nastiest. Now an expectation weighted by P(their action), from the same weights — `candidates` and `featuresFor` already take `side` . Across 44 boards `protectThreatened` fell 84%, `diesBeforeMoving` 78%. **The bot stops panicking.** Needs a refit before shipping, since nine features now mean something different.

Honest status / what it does NOT do — REWRITTEN 2026-07-30, because two of the four claims here had become false and were being quoted as current. It DOES now decide switches (voluntary switches score through `switchFeatures` ; the post-KO replacement is scored rather than rolled) and it DOES run a real damage calculation (`board.js` calls the damage engine throughout — `koTarget` , `killIsRoll` , `diesBeforeMoving` and the switch-survival features all read it). What remains true: it has **no model of the opponent's move**, so it cannot read a Protect or bait a switch; and it is **one ply, no search**. The weights are fitted on open-sheet games, which hedge less than closed ladder play, and **2.94%** of clicks could not be matched to a candidate and were dropped (`data/policy-weights.json` records it). This line used to read "**~11% ... mostly redirection (Follow Me, Rage Powder)**". **Both halves were wrong:** redirection is **1.60%** of the unmatched, measured 2026-08-02 by `engine/redirect_audit.js` , and the rate is now a quarter of what it was. The real causes were a foe **switching in on the same turn** (44.4%), an **in-battle forme change** with no sheet entry (19.7%), and a **mirror collapsing the two team sheets** (16.4%) — all fixed in `engine/click_match.js` , which took the slot-level match rate from 87.2% to 97.2%. Redirection's true cost is a *mislabelled* target on 1.55% of clicks, unrecoverable because the protocol records only a move's resolved target and never its chosen one. Logit also assumes independence of irrelevant alternatives, which close-substitute moves violate; see DEFENSE §6. **Corpus (as of 3.21.0):** three open-sheet sources, deduplicated by replay id, all through `quality.js` — `data/games.bo3.jsonl` (our own hourly scrape of `gen9championsvgc2026regmbbo3` , whose ruleset carries **Force Open Team Sheets**, so every game publishes all six sets), the ~1% of the closed ladder store where both players agreed to sheets, and the external VGC-Bench archive. **220,613 usable decisions kept of 228,084 seen**, from **8,414 games** (`data/policy-weights.json` , 2026-08-04; the line read 198,157 from 7,507 games at 2026-08-02, and 176,981 before `engine/click_match.js`). The 7,471 dropped are 6,669 unmatched, 776 trivial and 26 ambiguous, all recorded under `matching` . **Damage table:** 318 species. Eight had no row and therefore computed **zero damage, zero threat and zero risk** until 2026-08-02 — five format megas (Vitreobel, Feraligatr, Skarmory, Barbaracle, Falinks) whose data existed but was gated behind a stale `in_our_store` flag, plus Aegislash-Blade, Palafin-Hero and Gougeist's size formes, which need real rows because their stats differ from their base so the cosmetic fallback correctly refuses to substitute. `board.dmgMon.unknownSpecies` measured **6.16% → 0.00%** of candidate scorings; it had read 0.00% for weeks only because its workload was 120 replayed games, in which an in-battle forme change never reaches a scored position. **Covariate shift, corrected automatically:** open-sheet TEAMS differ from closed-sheet teams by 551.9 points of total absolute species difference (`engine/corpus_shift.js`), while measured behaviour given a board differs by at most 1.49. Every refit re-estimates on a sample reweighted to the closed-sheet species mix and reports the shift **in standard errors**. Five weights move materially — `priorLogP` 10.8 SE, `bp` 6.2 SE (sign flips) — so the **reweighted vector ships**, since MEW draws its teams from the ladder store. Board-reading weights (`eff` , `immune` , `deadStatus`) do not move. **Code:** `engine/board.js` , `engine/fit_policy.js` , `engine/magnemite.js` , `engine/corpus_shift.js` → `data/policy-weights.json` (both weight vectors, standard errors, and which shipped). Six assertions in `engine/selftest.js` under "board reading".

Job: decide by playing the position forward, instead of by scoring it once. MILTANK owns the bring, the lead, the mega timing and the post-KO replacement. **Why it is a separate model and not a MAG setting:** MAG scores an ACTION against the board in front of it. That is a one-ply question, and the four decisions above are not one-ply questions — a lead is a bet about turn three. Search is the only thing that can price it. **Named for Rollout**, which is the move that gets stronger the longer it is allowed to continue. The name is a description of the method, not a pun applied afterwards. **Result (R4, 2026-08-04):** beats MAG greedy on **55.5%** of **535 decisive pairs**, 95% CI [51.3, 59.7], over **2,624 games / 1,312 seed pairs**. SPRT accepted H_1 ($p=0.55$) at $\alpha=0.05$ after 522 decisive pairs. Artifact `data/rollout-r4.json` ; read it with `node engine/sprt.js data/games.r4-decided.jsonl` . **Three things that number is not, all recorded in the artifact:**

- **The n was wrong everywhere it was quoted until 2026-08-04.** `games.r4-decided.jsonl` holds **5,248 lines** and **2,624 games** — every id appears twice, a record plus a log-only companion. The generator now asserts the id-twice and seed-twice invariants and refuses to write without them.
- **The point estimate is biased high.** The run stopped at an SPRT boundary. The verdict carries the error rate; the 55.5% does not, and the CI is a fixed-n formula kept as context, not as inference.
- **No A/A noise floor exists for this comparison.** Three split-half cuts (spreads 0.2, 3.9 and 1.3 points against an effect of 5.5) stand in for one and are labelled a substitute, not a floor.

Standing: `PRE-CHANGE`. The engine source moved after the games were played, so transfer to the current build is an **assumption, not a result**. `player_digest.js` reports SAME PLAYER AS NOW — what moved was simulator mechanics, not the model — which is why the assumption is reasonable and still an assumption. **Earlier rungs:** R2 leaf cost (477 boards over 200 games), R3 divergence from MAG **72.9%** over 70 decisions, and **R1 — NOT ESTABLISHED**, corrected 2026-08-04.

R1's published PASS (9,201 positions, 68.18% against material's 65.26%, +2.91 [1.79, 4.04]) was prose only: `engine/rollout_r1.js` printed it and wrote no artifact, while `data/rollout-r1.json` held the *withdrawn* 230-row cross-language join and `engine/status.js` read that. Recomputed from the one committed input, `data/rollout-r1-rows.jsonl`, the gate is **UNDECIDED — rollout 65.72% against material's 65.26%, +0.46 points, 95% CI [-0.72, +1.63]** over the same 9,201 positions. The material column matches the published 65.26% exactly, so it is the same sample; the rollout column reproduces the *greedy* calibration table in `docs/ROLLOUT-design.md` §4.2.1 bin-for-bin, so the surviving dump is the `explore=0` incumbent and **the published 68.18% cannot be recomputed from anything committed**. The dump stamps no `N`, no `explore` and no build digest, which is why the two runs were indistinguishable; `rollout_r1.js` now writes `data/rollout-r1-rows.meta.json` beside every dump. Artifact `data/rollout-r1.json` (`engine/rollout_r1_artifact.js`); the withdrawn join is preserved at `data/rollout-r1-withdrawn-join.json` with `withdrawn: true`.

R2 and R3 had the same hole and were stamped 2026-08-04 through `engine/run_stamp.js`, one shared implementation rather than a third copy. Both published numbers reproduce as arithmetic and neither reproduction carries weight: **R3's 72.9% is $100 \times (70 - 19) / 70$ recomputed from two fields in its own file**, with no per-decision rows behind it, and **R2's timings cannot be recomputed by anyone** — a duration is a fact about a machine under a load and no per-leaf sample was dumped.

Two findings outrank the plumbing. **R3's noise floor was computed, printed and never written**, and the script's own verdict branches on it (`rate <= floor` → NOT A RESULT), so the committed artifact cannot say which branch its run took; the floors published in `docs/ROLLOUT-design.md` §5 belong to four earlier runs, and at N=20 the floor measured *higher* than the divergence. **R2 timed `explore=0` at `maxTurns=20`** by inheriting two library defaults, while MILTANK's in-game leaf is `explore=1.0` at `maxTurns=60` — the affordability table rests on the cost of a leaf the bot does not run. Separately, `data/rollout-r3.json`'s caveat claimed switches were excluded; commit `b4ec80b` put them on the menu and left the string alone. **Code:** `engine/miltank.js`, `engine/rollout_r4.js`, `engine/sprt.js`, `engine/paired_h2h.js`. Division ledger: `docs/SEARCH.md`. Paper: `docs/MILTANK.md`.

THE EIGHT ENTRIES BELOW WERE ADDED 2026-08-04 BECAUSE A GUARD SAID THEY WERE MISSING, NOT BECAUSE ANYONE REMEMBERED THEM. `tests/test-stadium-roster.js` grew a third direction — the set of things that actually GENERATE a `data/*` artifact, read out of `engine/provenance.js --graph` — and it went red on generators that appear in neither this ledger nor the Stadium. That is the GURU hole (PRIORITIES #41) reopening in eight more places, and it includes `data/meta-usage.json`, which `CLAUDE.md` itself calls "the model CHOMP reads", and `data/move-priors.json`, which nine files load. Every figure below is read out of the artifact named in its **Code** line.

Where a model has **no measured verdict**, it says **NOT MEASURED** rather than describing itself as working.

Job: say what this format actually plays — who is on teams, who gets brought, who leads, who wins — so that nothing downstream has to guess a prior. It is the file `CLAUDE.md` names as the CHOMP-facing model and the one `engine/mag_bot.js` carries into a live ladder game. **Method:** `engine/analyze.js` runs `quality.js` `loadGames()` over `data/games.ladder.jsonl` and tallies per-species `teamRate`, `bringRate`, `leadRate`, `winRate` and `n`. No model is fitted; it is a census. It publishes **two views and refuses to pick one:** `competitive` (bot-filtered — 7,123 clean games, 14,246 sampled teams, 239 species) and `ladder` (everything stored, bots included — 39,792 games, 52,966 teams, 261 species), with the file's own instruction that *"they answer different questions and neither is 'the' metagame."* **Corpus:** **7,123 clean of 39,792 collected**, generated **2026-08-04**, `provenance` **ok**. The funnel is carried in the artifact: 39,792 → 13,263 after the name-based bot rule → 10,480 after the behavioural one → 10,440 forfeits → 9,759 min-turns → **7,123 full-bring**. **Two facts that have to travel with it.** It was **24.1% behind the corpus until 2026-08-04**, when it was regenerated at Will's instruction — `docs/MEASURE.md` §5c had it filed **ASK** rather than **STOP** precisely because it types no figure into any document, and moving the live bot's meta prior is not a measurement pass's call to make alone. And it is **not** a refit trigger: `feature_fixture.js` excludes it by name and `board.js` never reads it. **Honest status:** the numbers are current and the population is declared, which is more than most artifacts here manage. What is **NOT MEASURED** is whether acting on them helps — no result in this repository compares a decision made with this prior against one made without it. **Its own stated limit, quoted rather than paraphrased:** *"Bot detection is name-based plus a team-invariance rule. Accounts that play few games or vary their team can still escape it. Describe this set as 'no bot detected', not as human."* **Code:** `engine/analyze.js` → `data/meta-usage.json`. Read by `engine/mag_bot.js` (the live bot), `engine/mew.js`, `engine/kadabra.js`, `engine/ditto.js`, `engine/coach.js`, `engine/chomp_ev.js`, CHOMP, the Tower's threat-list bundle, `app/models.html` and `app/tower.html`.

Job: answer "what does this species actually CLICK", so that a rollout picks Tailwind, Fake Out, Spore, Swords Dance and Protect because strong players do, instead of only ever picking max damage. **Method:** `engine/policy.js` walks the per-turn event stream of clean ladder games and counts every move a species was seen to use. It publishes, per species, the **top 8 moves by P(move | action)** tagged with `kind / effect / boosts` from `moves-meta.js`, plus a separate **top-4 turn-1 lead** distribution. Species with fewer than 15 recorded actions are dropped. **Corpus:** **295 species, 128,548 recorded clicks** (median 147 per species, range 15–5,438), generated **2026-07-31** — the 5,269-clean-game era, so it is roughly **26% behind** the store. **IT DECLARES NO GAME COUNT, so the drift check cannot see that;** `provenance.js` reports only *"records no game count — nobody can check what it was built from."* Fixing that is a one-line change in the generator and it is not made here. **THE GENERATOR IN THIS LINE IS NOT THE ONE THE GRAPH USED TO NAME, and that is the reason this entry exists.** `engine/provenance.js` credited `data/move-priors.json` to `engine/state_encoder.py`, which only **reads** it. The real writer is `engine/policy.js`, and it was invisible because the JavaScript spelling of the path-indirection idiom (`const OUT = process.argv[3] || path.join(...)`) put `const` where the checker expected an identifier. The cost was not a label: `state_encoder.py` opens no game file, so the artifact was classed **not store-derived and was exempt from every corpus check in the repository**. Fixed 2026-08-04; the same pass made seven more artifacts visible. **Honest status:** it is a measured frequency table and nothing more. **NOT MEASURED:** whether sampling from it produces more human-like play than any alternative. Its quantity is also **not** the one Smogon publishes — this is **P(move | action)**, Smogon's is **P(move is ON the set)** — and `engine/set_priors.js` prefers Smogon's for set inference for exactly that reason. **Code:** `engine/policy.js` → `data/move-priors.json`. Read by `engine/board.js`, `engine/medicham2-browser.js`, `engine/rollout_leaf.js`, `engine/fit_policy.js`, `engine/prior_player.js`, `engine/mew.js`, `engine/set_priors.js`, `engine/smogon_priors.js`, and the MAG browser bundle `data/mag.js`.

Job: score a POSITION — given a board with no action attached, how likely is this side to win. It is the other candidate leaf beside `rolloutWinProb`, and the model `docs/POKER-TO-POKEMON.md §7` proposes training on self-play and calibrating on humans. **Method:** `engine/porygon2.py` embeds each position in **17 features** — `material ( alive_diff , hp_active_diff , hp_total_diff , my_alive , foe_alive , turn )`, `state ( status_diff , boost_diff , tailwind_diff , screen_diff , hazard_diff , trickroom , weather_on , terrain_on )` and `matchup ( matchup_edge , speed_edge , type_threat )` — and looks up the k nearest neighbours. No weights are fitted; it is a lookup. **Corpus: trained on 3,898 self-play games / 73,368 positions, evaluated on 2,274 held-out clean HUMAN ladder games / 12,000 positions.** Generated **2026-07-28**, `provenance` **ok**.

arm	accuracy	Brier	log-loss
coin	50.22%	0.2500	0.6931
material sign — the baseline that matters	60.22%	0.2254	0.6410
plain k=200	62.61%	0.2194	0.6258
weighted k=50	63.59%	0.2239	0.6483

Honest status: it beats material by about 2.4 accuracy points and 0.015 of log-loss, and NOT ONE OF THOSE NUMBERS HAS AN INTERVAL. The artifact publishes six point estimates and no CI, no paired test and no split-half floor, so *"beats material"* is currently an ordering of six numbers, not a result. The smallest split-half floor this project has published is 0.43 points. **NOT MEASURED.** Note also that its best accuracy and its best log-loss come from **different arms**, which is what a table with no uncertainty on it looks like. **Its own stated caveat, quoted:** *"Self-play positions come from MAG playing itself. If MAG's play is unlike human play the neighbourhoods are unrepresentative and the lookup is confidently wrong."* That is why the evaluation set is human, and it is the right design. **And CLAUDE.md's own charge stands against it:** *"PORYGON2 and DODUO were fitted, saved, quoted in documents, and never once in a live decision."* It is read by `engine/mew.js` and `engine/player_digest.js` and by two GATE scripts; **no live decision calls it.** **Code:** `engine/porygon2.py` → `data/porygon2.json`, `data/porygon2-curve.json` (the k-sweep) and `data/porygon2-species.json` (types, Speed and the type chart exported from `data/engine-data.js` so there is no second copy of the dex). Distinct from PORY (`engine/pory.py`), the logistic value net retracted 2026-08-02 — see `docs/MEASURE.md §5d`.

Job: replace the one fictional set per species that `data/engine-data.js` inherited with the **joint** sets real players declare, so the Battle Tower, the rollouts and DITTO's referee stop building Pokemon nobody plays. **Method:** `engine/derive_sets.js` reads the OPEN-SHEET corpora — `games.bo3.jsonl` and `games.ots.jsonl`, through `quality.js clean` — and tallies the exact (item, ability, nature, moves) tuple as one unit, with counts and shares. **Joint, not marginal, and that is the whole point:** Will's case was Sneasler, where Gunk Shot and Dire Claw are each common and few sets carry both, so the top-four-by-marginal is a set nobody runs. Measured, Garchomp's top-four marginals are the real set only **43.7%** of the time. **Corpus: 247 species over 7,175 open-sheet games and 85,992 sheet entries**, generated **2026-08-02**, `provenance` **ok**. The median species has **19 distinct sets** with the most common holding **25%**; Garchomp has **4,990 sheets, 311 distinct sets**, top set Life Orb / Earthquake / Dragon Claw / Rock Slide / Protect at **33.8%**. **Honest status:** this is DECLARED data, not observed — it is what a sheet said at preview, and `CLAUDE.md`'s *PREFER OBSERVED OVER DECLARED* applies the moment a Knock Off or a Trick lands. It is also an **open-sheet** population, which is not the ladder: only ~1% of ladder games carry a sheet (`docs/MEASURE.md §16a`). **NOT MEASURED:** whether building from these sets changes any outcome. The distribution is a fact; the improvement is a claim nobody has tested. **Code:** `engine/derive_sets.js` → `data/species-sets.json`. Read by `engine/showdown_bot.js`, `engine/species_sets.js` and `build/rebuild_sets_from_sheets.js`.

Job: answer the question a player actually asks. Not *"who wins"* — that is GURU — but *"what beats this"*: of the games where somebody beat a Rain team, what did the winner bring that the loser did not. **Method:** `engine/counters.py` fixes the opposing archetype and compares **P(species brought | won)** against **P(species brought | lost)** inside that matchup, with a Wilson interval on each side and a two-proportion z on the difference, then a **Benjamini–Hochberg** correction across all tests. Pooling on a feature WITHIN a matchup instead of splitting into a cell per pair is the design argument, and it is a good one: every game against Rain contributes to every species' count. **Corpus: 11 opposing archetypes, 512–4,467 games each, 1,081 tests**, generated **2026-08-04**, thresholds `min_games` 40 / `min_seen` 8. **THE HEADLINE IS A NULL, and it has to be said in that order. 61 of 1,081 tests clear a nominal 95% z. ZERO survive Benjamini–Hochberg in any of the eleven matchups.** 5% of 1,081 is 54, and 61 were found. **There is no species this file can say counters anything.** `data/counters.json` **was UNSAFE — older than the quality filter — for NINE DAYS**, meaning it was computed under a different definition of which games count, and nothing could see it: the artifact was invisible to `engine/provenance.js` until the loader-call fix of 2026-08-03 made it visible, at which point it failed immediately and was regenerated (15 s). That is the check working, and it is the reason this ledger records the artifact's status and not just its verdict. **Its own stated caveat, quoted:** *"brought is what the replay REVEALED and conditions on game length; species are correlated so lift is marginal, not causal."* **Honest status: computed, correct, and consumed by NOTHING.** No engine, page or bot reads `data/counters.json`. Per this project's own rule an unwired model is still a model — consumption is evidence about importance, never about whether something needs writing down. **Code:** `engine/counters.py` → `data/counters.json`.

Job: give the opponent model a defensible draw. Before a game reveals anything, what is the chance this species is on the field at turn 1, and what is the chance it is brought at all. **Method:** `engine/bring_priors.js` measures `p_lead` from turn-1 leads and `p_bring` from revealed brings over clean games, shrunk toward the pool mean by **10 pseudo-observations** so a species seen four times cannot land at 1.00 and dominate every draw. It also measures the format's mega rates off the raw protocol logs. **Corpus: 14,456 sides, 277 species**, regenerated **2026-08-04**. Pool means `bring` **66.7%**, `lead` **50.0%**. **IT WAS UNSAFE — five minutes older than the quality filter — AND NOBODY COULD SEE IT.** The artifact was absent from `provenance.js`'s graph entirely until 2026-08-04, for the same `const OUT = ...` reason as MOVE PRIORS above. Regenerating it moved one figure a long way and it is worth recording: the **mega rate was measured on 62 sides and is now measured on 12,442**, and `p_side_megas` moved **0.9355 → 0.8785** with `p_mega_is_lead` **0.5345 → 0.5159**. `CLAUDE.md` sets a domain RATE floor here — *"a game without a mega should be rare"* — and the floor was being checked against a 62-side sample. **Honest status:** `p_lead` rests on solid ground; `p_bring` **does not, and the generator says so first.** A Pokemon selected but never sent out is invisible to a replay, so `p_bring` is biased down, hardest for slow and situational mons in short games. **Treat it as a ranking, not a calibrated rate. NOT MEASURED:** whether an opponent drawn from these priors is closer to a real opponent than a uniform draw. **Code:** `engine/bring_priors.js` → `data/bring-priors.json`. Read by `engine/mew.js`, `engine/prior_player.js` and `engine/selftest.js`.

Job: GURU's question one level finer. Not *"does archetype A beat archetype B"* but *"does this PAIR beat that pair"*, where the pair is the $C(4,2) = 6$ combinations inside a side's real bring. **Method:** `engine/cores.js` enumerates every pair inside each clean side's bring, keeps the most common **K = 24** cores, and builds a core × core matrix from real outcomes with **Wilson 95% intervals**, deliberately in the same shape as `data/guru-matchups.json` so SLOWKING and the Hodge decomposition can consume it unchanged. **Corpus: 5,265 clean games, 24 cores, 552 populated directed cells**, generated **2026-07-31** and **27.2% behind** the 7,228 clean ladder games now available. The most common core is Archaludon + Pelipper at 528 brings. **THE FINER GRAIN IS THE PROBLEM, NOT THE FEATURE. The median cell rests on 9 games** (max 33), and **32 of 552 cells** have an interval excluding 50% — **5.8% against the 5% a nominal 95% interval produces by construction when nothing is true.** GURU splits 5,265 games into 144 cells and

finds nothing that survives multiplicity; this splits the same games into 552 and is thinner still. **NOT MEASURED, and on this corpus it structurally cannot be:** no multiplicity correction is applied and none would leave anything standing. **Honest status:** the right shape, computed correctly, on far too little data per cell. It is read only by `engine/sanity_check.py`. **Do not quote a cell from it. Code:** `engine/cores.js` → `data/core-matchups.json` (CORES env sets K).

Job: two things the usage model cannot see, both measured off the event stream rather than assumed. **Who actually moved first, and what each move actually took off. Method:** `engine/dynamics.js` walks the stored turns. For every non-priority, non-Trick-Room exchange it records which side resolved first, aggregates to a per-species `firstRate`, and raises a `scarfHint` when a species outspeeds what its base stat allows. Separately it records the observed damage percentage per (attacker, move) — the real roll distribution, with mean, max and p90. **Corpus: 3,665 games (2,635 of them non-Trick-Room), 240 species with a speed signal, 1,531 (attacker, move) damage pairs**, generated **2026-07-28. Honest status: a census, and its two halves are not equally trustworthy.** The damage half is a distribution of observed rolls and is as good as its sample. The speed half infers a HIDDEN quantity — a spread and an item — from an ordering, and `scarfHint` is a heuristic with **no measured false positive rate:** Aegislash is flagged at `firstRate` 0.939 on **n = 33**, which is also consistent with Aegislash being paired with Trick Room or with slow opponents. **NOT MEASURED.** The engine's own facts about speed live in `engine/board.js` and `engine/medicham2-browser.js` and are NOT taken from this file, which is correct — this is evidence, not a rule. **Code:** `engine/dynamics.js` → `data/dynamics.json`. Read by `engine/ditto.js`, `engine/kadabra.js`, `engine/jolteon.py` and `engine/slowking.py`.

Job: be the rules authority, replacing a hand-written engine that was wrong in eight silent ways. **Status:** wired and **verified.** `engine/validate_damage_sim.js` runs the 31-scenario golden master through the official engine against `@smogon/calc` : **31/31 within 2%**. That clears ADR-001 migration step 3. **Why the check mattered:** it is a test of OUR WIRING, not of Showdown. ADR-001 records four engine comparisons of which three produced confident wrong numbers from mis-wiring, none of which crashed. It caught two more on its first run — a forced maximum roll that also forced a critical hit, and the discovery that `battle.randomChance()` bypasses `battle.random()` entirely. **Speed:** 29 battles/sec/core against the hand-written engine's 3,401. Offline only; the browser must never simulate. **Code:** `engine/champions_sim.js`, pinned commit `20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4`.

Job: replace three things ABRA was guessing with measurements over the whole ladder. **What it corrected:** every Pokémon had a flat `11/11/11/11/11/11` SP spread. Real Garchomp runs `Jolly 2/32/0/0/0/32` on 42% of sets, so Attack was 161 where it should be 182 — a 13% understatement of the format's most-used attacker, in every damage figure the project had produced. It also supplies items and abilities where the closed-sheet store has 69.7% and 75.5% unknown, and **P(move is ON the set)** where our behaviour-clone measures the different quantity **P(move | action)**. **Confirmed two mechanics independently:** the SP budget is 66 (97% of real spreads spend all of it) and SP is capped at **32 per stat** (92% of spreads touch it). **Limit:** aggregate. Describes the population, never a game, and cannot be joined to a replay. **Code:** `engine/fetch_smogon_stats.js` (archived monthly by CI), `engine/smogon_priors.js` → `data/smogon-priors.json`.

Every model below is fitted or measured under conditions that must be stated with the number.

- **A measurement reads a frozen release, not the live tree.** `engine/engine_release.js`; current release `5fc1f711a0e3` over 12 files including the weights. A number produced without a release stamp cannot be reproduced, because the tree it was scored against is not recoverable.

- `data/exploitability.json` **is** `void: true` **and** `provenance.js` **reads it UNSAFE**. MAG has **no exploitability figure**. The 63.2% [56.6, 69.3] this ledger once called the most important number in the repository is retracted: 17 features against 58, an engine 25 wire-fixes old, computed before the quality filter existed.
- **MAG's weights moved, and it changed nothing measurable**. The board weather defect was real (14 of 58 columns, 10.72% of turn-boards) and worth fixing on its own terms; refitting on top of it returned an interval containing zero on both metrics.
- **MAG is fitted without the ability and moves its live player reads**. 50.47% of the decisions it trains on are priced against a board `magnemite.js` does not present. Unfixed by decision — it requires a full refit and a prior answer about opponents who decline open team sheets.